```
# ===== edge/arm_bridge/main.py =====
#!/usr/bin/env python
"""Run the arm bridge against a real broker or as a no-hardware self test.
Examples:
    # No broker, no hardware: inject one dispatch and print the full loop.
    python edge/arm_bridge/main.py --dry-run
    # Verify that a named driver resolves from the registry + config.
    python edge/arm_bridge/main.py --check-driver --driver dobot_example
    # Real broker with the configured driver (simulated or HTTP).
     python edge/arm_bridge/main.py --config edge/arm_bridge/config.yaml
The self test proves the MQTT contract only (E1/E2). It is not physical
pickup evidence and does not imply detection accuracy.
 """
from __future__ import annotations
import argparse
import signal
import sys
import time
from pathlib import Path
from typing import Any
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
if str(EDGE) not in sys.path:
    sys.path.insert(0, str(EDGE))
import yaml  # noqa: E402
from arm_bridge.bridge import ArmBridge  # noqa: E402
from arm_bridge.drivers import build_arm_driver  # noqa: E402
from arm_bridge.mqtt_transport import MqttTransport  # noqa: E402
from device_sim.protocol import FakeClock  # noqa: E402
from device_sim.transport import MemoryTransport  # noqa: E402
DEFAULT_CONFIG = HERE / "config.yaml"
def _load_config(path: Path) -> dict[str, Any]:
    return yaml.safe_load(Path(path).read_text(encoding="utf-8")) or {}
def build_driver(cfg: dict[str, Any], backend: str | None) -> Any:
    driver_cfg = cfg.get("driver", {})
    return build_arm_driver(driver_cfg, backend)
def run_self_test(cfg: dict[str, Any]) -> int:
    """Run one dispatch through an in-memory transport and exit."""
    # Deliberately simulated: the self test proves the MQTT contract without
    # touching a physical arm or a vendor HTTP endpoint. Driver resolution is
    # covered separately by --check-driver.
    transport = MemoryTransport()
    bridge = ArmBridge(
        device_id=str(cfg["device_id"]),
        site_id=str(cfg["site_id"]),
        driver=build_driver(cfg, "simulated"),
        transport=transport,
    )
    bridge.start()
    now = time.time()
```

```
    payload = {
        "command_id": "cmd_tsk_selftest_0001",
        "device_id": cfg["device_id"],
        "seq": 1,
        "action": "dispatch",
        "issued_at": now,
        "issued_at_iso": FakeClock.iso(now),
        "expires_at": now + 30.0,
        "expires_at_iso": FakeClock.iso(now + 30.0),
        "params": {
            "task_id": "tsk_selftest_0001",
            "target": {"lng": 119.6531, "lat": 26.3867},
            "priority": 1,
        },
    }
    transport.deliver(bridge.task_topic, payload, qos=1)
    print("[self-test] command accepted")
    for topic, body, qos in transport.published:
        print(f"[self-test] {topic} qos={qos}")
        print(f"            {body}")
    statuses = [p["status"] for p in bridge.history["progress"]]
    if statuses != ["collecting", "done"]:
        print(f"[self-test] FAIL: unexpected progress sequence {statuses}", file=sys.stderr)
        return 1
    if bridge.counter["pick_executed"] != 1:
        print("[self-test] FAIL: driver pick did not execute", file=sys.stderr)
        return 1
    print("[self-test] OK: assigned -> navigating(ack) -> collecting -> done")
    return 0
def check_driver(cfg: dict[str, Any], backend: str | None) -> int:
    """Build the named driver from registry + config without calling it."""
    try:
        driver = build_driver(cfg, backend)
    except Exception as exc:  # noqa: BLE001
        print(f"[check-driver] FAIL: {exc}", file=sys.stderr)
        return 1
    print(f"[check-driver] OK: backend={backend or cfg.get('driver', {}).get('backend', 'simulated'…
    return 0
def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--config", default=str(DEFAULT_CONFIG))
    parser.add_argument("--dry-run", action="store_true", help="no broker; run self test")
    parser.add_argument("--self-test", action="store_true", help="alias for --dry-run")
    parser.add_argument(
        "--check-driver",
        action="store_true",
        help="resolve driver from registry + config, without calling it",
    )
    parser.add_argument("--driver", default=None, help="override driver.backend")
     parser.add_argument("--device-id", default=None)
```

```
    parser.add_argument("--site-id", default=None)


    return parser.parse_args(argv)
def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    cfg = _load_config(Path(args.config))
    if args.check_driver:
        return check_driver(cfg, args.driver)
    if args.dry_run or args.self_test:
        return run_self_test(cfg)
    device_id = args.device_id or str(cfg["device_id"])
    site_id = args.site_id or str(cfg["site_id"])
    mqtt_cfg = cfg.get("mqtt", {})
    transport = MqttTransport(
        host=str(mqtt_cfg.get("host", "localhost")),
        port=int(mqtt_cfg.get("port", 1883)),
        username=mqtt_cfg.get("username"),
        password=mqtt_cfg.get("password"),
        client_id=f"robot-{device_id}",
        keepalive=int(mqtt_cfg.get("keepalive", 60)),
        use_lwt=bool(mqtt_cfg.get("use_lwt", True)),
        will_topic=f"marine/{site_id}/{device_id}/status",
    )
    bridge = ArmBridge(
        device_id=device_id,
        site_id=site_id,
        driver=build_driver(cfg, args.driver),
        transport=transport,
    )
    bridge.start()
    try:
        transport.connect()
    except Exception as exc:  # noqa: BLE001
        print(f"[error] cannot connect MQTT broker: {exc}", file=sys.stderr)
        print("        use --dry-run to run the self test without a broker", file=sys.stderr)
        return 2
    bridge.publish_online()
    print(f"[arm-bridge] {device_id} at {site_id} connected, waiting for tasks")
    stop = False
    def _stop(_signum: int, _frame: Any) -> None:
        nonlocal stop
        stop = True
    signal.signal(signal.SIGINT, _stop)
    signal.signal(signal.SIGTERM, _stop)
    while not stop:
        time.sleep(1.0)
    bridge.publish_offline()
    transport.disconnect()
    print("[arm-bridge] stopped")
    return 0
if __name__ == "__main__":
```

```
    raise SystemExit(main())



# ===== edge/arm_bridge/bridge.py =====
"""MQTT contract bridge between Oceanus and any physical arm.
Responsibilities:
- Subscribe to robot/{id}/task and robot/{id}/cmd.
- Acknowledge commands with the frozen ACK envelope.
- Translate arm driver results into task/progress and marine/...
  telemetry messages.
- Deduplicate commands by command_id and reject expired commands.
The bridge never talks to arm-specific SDKs directly. Implement one
ArmDriver adapter per arm model and the rest of the chain stays the same.
Evidence level: E1/E2. Passing self tests proves the protocol loop under a
deterministic transport, not physical pickup or field acceptance.
"""
from __future__ import annotations
import logging
import sys
import time
from pathlib import Path
from typing import Any
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
if str(EDGE) not in sys.path:
    sys.path.insert(0, str(EDGE))
from device_sim.protocol import (  # noqa: E402
    COMMAND_ACTIONS,
    Ack,
    Clock,
    DeviceCommand,
    DeviceMode,
    FakeClock,
    IdFactory,
    Position,
    SequentialIdFactory,
  )
from device_sim.transport import DeviceTransport, MemoryTransport  # noqa: E402
from .drivers import ArmDriver, DriverError, PickResult
logger = logging.getLogger("arm_bridge")
class WallClock:
    """Real-time clock implementing the injectable Clock protocol."""
    def now(self) -> float:
        return time.time()
def _iso(epoch: float) -> str:
    return FakeClock.iso(epoch)
class ArmBridge:
    """Reusable bridge connecting one robot device to the platform."""
    def __init__(
        self,
        device_id: str,
        site_id: str,
```

```
        driver: ArmDriver,
        transport: DeviceTransport | None = None,
        *,
        clock: Clock | None = None,
        id_factory: IdFactory | None = None,
    ) -> None:
        self.device_id = str(device_id)
        self.site_id = str(site_id)
        self.driver = driver
        self.transport = transport or MemoryTransport()
        self.clock = clock or WallClock()
        self.id_factory = id_factory or SequentialIdFactory("ack")
        self.task_topic = f"robot/{self.device_id}/task"
        self.cmd_topic = f"robot/{self.device_id}/cmd"
        self.ack_topic = f"robot/{self.device_id}/cmd/ack"
        self.progress_topic = f"robot/{self.device_id}/task/progress"
        self.telemetry_topic = f"marine/{self.site_id}/{self.device_id}/telemetry"
        self.status_topic = f"marine/{self.site_id}/{self.device_id}/status"
        self._started = False
        self._acked: dict[str, Ack] = {}
        self.history: dict[str, list[Any]] = {
            "commands": [],
            "acks": [],
            "progress": [],
            "telemetry": [],
        }
        self.events: list[dict[str, Any]] = []
        self.counter: dict[str, int] = {
            "commands_processed": 0,
            "commands_invalid": 0,
            "commands_rejected": 0,
            "duplicates": 0,
            "acks_published": 0,
            "progress_published": 0,
            "telemetry_published": 0,
            "pick_executed": 0,
            "pick_failed": 0,
        }
    # ------------------------------------------------------------------
    # Lifecycle
    # ------------------------------------------------------------------
    def start(self) -> None:
        """Subscribe to the robot command channels."""
        if self._started:
            return
        self._started = True
        self.transport.subscribe(self.task_topic, self._on_message)
        self.transport.subscribe(self.cmd_topic, self._on_message)
    def stop(self) -> None:
        if not self._started:
```

```
            return


        self._started = False
        self.transport.unsubscribe(self.task_topic, self._on_message)
        self.transport.unsubscribe(self.cmd_topic, self._on_message)
    def publish_online(self) -> None:
        now = self.clock.now()
        self.transport.publish(
            self.status_topic,
            {"device_id": self.device_id, "online": True, "ts": _iso(now)},
            qos=1,
        )
    def publish_offline(self) -> None:
        now = self.clock.now()
        self.transport.publish(
            self.status_topic,
            {"device_id": self.device_id, "online": False, "ts": _iso(now)},
            qos=1,
        )
    def receive_command(self, command: DeviceCommand | dict[str, Any]) -> None:
        """Inject a command directly (tests and self-test mode)."""
        if isinstance(command, dict):
            command = DeviceCommand.from_dict(command)
        self._handle_command(command, self.clock.now())
    def snapshot(self) -> dict[str, Any]:
        status = self.driver.status()
        return {
            "device_id": self.device_id,
            "site_id": self.site_id,
            "mode": status.mode,
            "counter": dict(self.counter),
            "events": list(self.events),
        }
    # ------------------------------------------------------------------
    # Inbound commands
    # ------------------------------------------------------------------
    def _on_message(self, topic: str, payload: dict[str, Any]) -> None:
        try:
            command = DeviceCommand.from_dict(payload)
        except (KeyError, TypeError, ValueError) as exc:
            self.counter["commands_invalid"] += 1
            self._event("command_invalid", topic=topic, error=str(exc))
            return
        self._handle_command(command, self.clock.now())
    def _handle_command(self, command: DeviceCommand, now: float) -> None:
        self.counter["commands_processed"] += 1
        self.history["commands"].append(command)
        if command.device_id != self.device_id:
            self.counter["commands_invalid"] += 1
            self._event(
                "command_invalid",
```

```
            command_id=command.command_id,
            reason="device_id_mismatch",
        )
        return
    previous = self._acked.get(command.command_id)
    if previous is not None:
        self.counter["duplicates"] += 1
        self._event("duplicate_command", command_id=command.command_id)
        self._publish_ack(previous)
        return
    if command.action not in COMMAND_ACTIONS:
        self._reject(command, "unknown_action", now)
        return
    if command.is_expired(now):
        self._reject(command, "expired", now)
        return
    if command.action == "dispatch":
        self._handle_dispatch(command, now)
        return
    self._handle_other(command, now)
def _handle_dispatch(self, command: DeviceCommand, now: float) -> None:
    task_id = command.params.get("task_id")
    target = command.params.get("target")
    if not task_id or not isinstance(target, dict):
        self._reject(command, "missing_target", now)
        return
    try:
        target_pos = Position(float(target["lng"]), float(target["lat"]))
    except (KeyError, TypeError, ValueError):
        self._reject(command, "missing_target", now)
        return
    ack = self._make_ack(
        command, accepted=True, reason="dispatched", mode=DeviceMode.NAVIGATING, now=now
    )
    self._publish_ack(ack)
    self._event("dispatch_accepted", command_id=command.command_id, task_id=task_id)
    self._publish_progress(task_id, DeviceMode.COLLECTING)
    self._publish_telemetry(
        status=DeviceMode.COLLECTING,
        task_id=task_id,
        location=target_pos,
        now=now,
    )
    priority = int(command.params.get("priority", 3))
    try:
        result = self.driver.pick(task_id, target_pos, priority)
        self.counter["pick_executed"] += 1
        self._event("pick_done", command_id=command.command_id, task_id=task_id)
    except DriverError as exc:
        self.counter["pick_failed"] += 1
```

```
            self._event("pick_failed", command_id=command.command_id, error=str(exc))

            result = PickResult(
                ok=False,
                collected_weight=0.0,
                review_result="recheck",
                bins_after=dict(self.driver.status().bins),
            )
        self._publish_progress_done(task_id, result, now)
        self._publish_telemetry(
            status=DeviceMode.IDLE,
            task_id=task_id,
            location=self.driver.status().location,
            now=now,
        )
    def _handle_other(self, command: DeviceCommand, now: float) -> None:
        action = command.action
        accepted = True
        reason = {
            "pause": "paused",
            "resume": "resumed",
            "return_home": "returning",
            "emergency_stop": "emergency_stop",
            "ack": "ack_echo",
        }.get(action, "accepted")
        try:
            if action == "pause":
                self.driver.pause()
            elif action == "resume":
                self.driver.resume()
            elif action == "return_home":
                self.driver.return_home()
            elif action == "emergency_stop":
                self.driver.emergency_stop()
        except DriverError as exc:
            accepted = False
            reason = "driver_error"
            self._event("driver_error", command_id=command.command_id, error=str(exc))
        ack = self._make_ack(
            command,
            accepted=accepted,
            reason=reason,
            mode=self.driver.status().mode,
            now=now,
        )
        self._publish_ack(ack)
        self._event("command_applied", command_id=command.command_id, action=action)
    # ------------------------------------------------------------------
    # Outbound protocol messages
    # ------------------------------------------------------------------
    def _reject(self, command: DeviceCommand, reason: str, now: float) -> None:
```

```
        self.counter["commands_rejected"] += 1



        ack = self._make_ack(
            command,
            accepted=False,
            reason=reason,
            mode=self.driver.status().mode,
            now=now,
        )
        self._publish_ack(ack)
        self._event("command_rejected", command_id=command.command_id, reason=reason)
    def _make_ack(
        self,
        command: DeviceCommand,
        *,
        accepted: bool,
        reason: str,
        mode: str,
        now: float,
    ) -> Ack:
        return Ack(
            ack_id=self.id_factory.next(),
            command_id=command.command_id,
            device_id=self.device_id,
            seq=command.seq,
            received_at=now,
            accepted=accepted,
            reason=reason,
            mode=mode,
        )
    def _publish_ack(self, ack: Ack) -> None:
        self._acked[ack.command_id] = ack
        self.transport.publish(self.ack_topic, ack.to_dict(), qos=1)
        self.history["acks"].append(ack)
        self.counter["acks_published"] += 1
        self._event("ack_published", command_id=ack.command_id, accepted=ack.accepted)
    def _publish_progress(self, task_id: str, status: str) -> None:
        payload = {
            "task_id": task_id,
            "robot_id": self.device_id,
            "status": status,
            "progress": 1.0 if status == "done" else 0.5,
            "ts": _iso(self.clock.now()),
        }
        self.transport.publish(self.progress_topic, payload, qos=1)
        self.history["progress"].append(payload)
        self.counter["progress_published"] += 1
        self._event("progress_published", task_id=task_id, status=status)
    def _publish_progress_done(
        self, task_id: str, result: PickResult, now: float
    ) -> None:
```

```
        payload = {
            "task_id": task_id,
            "robot_id": self.device_id,
            "status": "done",
            "progress": 1.0,
            "collected_weight": round(float(result.collected_weight), 3),
            "review_result": result.review_result,
            "evidence_url": result.evidence_url,
            "bins_after": result.bins_after,
            "ts": _iso(now),
        }
        self.transport.publish(self.progress_topic, payload, qos=1)
        self.history["progress"].append(payload)
        self.counter["progress_published"] += 1
        self._event("progress_done", task_id=task_id, review_result=result.review_result)
    def _publish_telemetry(
        self,
        *,
        status: str,
        task_id: str | None,
        location: Position,
        now: float,
    ) -> None:
        driver_status = self.driver.status()
        payload = {
            "device_id": self.device_id,
            "device_type": "robot",
            "timestamp": _iso(now),
            "location": location.to_dict(),
            "status": status,
            "battery": driver_status.battery,
            "bins": dict(driver_status.bins),
            "task_id": task_id,
            "speed": driver_status.speed,
            "heading": driver_status.heading,
            # 逐舵机遥测（电压/温度/位置）。这是"机械臂真的动了"的
            #   硬证据：status 字段是平台自报，这里的数字来自舵机本身。
            #   采集不到时是空 dict，不会让整条消息失败。
            "servos": dict(getattr(driver_status, "servos", {}) or {}),
        }
        self.transport.publish(self.telemetry_topic, payload, qos=0)
        self.history["telemetry"].append(payload)
        self.counter["telemetry_published"] += 1
        self._event("telemetry_published", status=status, task_id=task_id)
    def _event(self, kind: str, **detail: Any) -> None:
        self.events.append({"kind": kind, "detail": detail})
__all__ = ["ArmBridge", "WallClock"]
# ===== edge/arm_bridge/drivers.py =====
"""Arm driver adapters.
ArmDriver is the only interface a new arm has to implement. The bridge and
```

```
the platform MQTT contract stay unchanged when hardware changes.



Evidence level: E1/E2. Simulated and generic HTTP drivers are deterministic
prototype adapters; they do not prove physical pickup or field acceptance.
"""
from __future__ import annotations
import importlib
import json
import sys
import time
import urllib.error
import urllib.request
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Callable, Protocol
from device_sim.protocol import DeviceMode, Position
class DriverError(RuntimeError):
    """Raised when the underlying arm SDK rejects or fails an action."""
@dataclass(frozen=True)
class PickResult:
    """Outcome of one pickup action, translated to the platform contract."""
    ok: bool = True
    collected_weight: float = 0.0
    review_result: str = "confirmed"
    evidence_url: str | None = None
    bins_after: dict[str, float] = field(
        default_factory=lambda: {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
    )
@dataclass(frozen=True)
class ArmStatus:
    """Current status snapshot reported as marine telemetry."""
    mode: str = DeviceMode.IDLE
    battery: int = 100
    bins: dict[str, float] = field(
        default_factory=lambda: {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
    )
    location: Position = Position(119.6540, 26.3870)
    heading: float = 0.0
    speed: float = 0.0
    #: Per-servo telemetry read back from the arm, keyed by servo id.
    #: 证据链价值：电压/温度/位置是"机械臂真的动了"的硬证据，
    #:   比 progress 报文自报更有说服力。None 表示该次读取失败。
    servos: dict[str, Any] = field(default_factory=dict)
class ArmDriver(Protocol):
    """Adapter contract for an arbitrary arm SDK.
    Implement this class once per physical arm model. The bridge calls these
    methods synchronously and translates results into MQTT messages.
    """
    def status(self) -> ArmStatus: ...
    def pick(self, task_id: str, target: Position, priority: int) -> PickResult: ...
    def pause(self) -> None: ...
```

```
    def resume(self) -> None: ...



    def return_home(self) -> None: ...
    def emergency_stop(self) -> None: ...
#: 需要额外运行时依赖的驱动 -> 模块路径。惰性加载，见 build_arm_driver。
_OPTIONAL_DRIVER_MODULES = {
    "ros_arm_control": "arm_bridge.ros_driver",
 }
ArmDriverFactory = Callable[..., Any]
ARM_DRIVER_REGISTRY: dict[str, ArmDriverFactory] = {}
def register_arm_driver(name: str, factory: ArmDriverFactory) -> None:
    """Register a named arm driver so config can select it without code changes."""
    driver_name = str(name).strip()
    if not driver_name:
        raise ValueError("driver name must not be empty")
    ARM_DRIVER_REGISTRY[driver_name] = factory
def build_arm_driver(
    driver_cfg: dict[str, Any],
    backend: str | None = None,
) -> Any:
    """Build a driver from the driver config section.
    Supported layouts:
    * backend: simulated -> SimulatedArmDriver
    * backend: http -> HttpArmDriver
    * backend: <brand> with a <brand>: {...} block -> registered class
    * backend: <brand> with vendors.<brand>: {kind, ...} -> registered
      class selected by kind
    """
    effective_backend = str(backend or driver_cfg.get("backend", "simulated"))
    # 可选驱动惰性加载：ros_driver 只在真正选用它时才 import。
    #   原因：它 import rospy，而 rospy 只在树莓派上有；开发机与 CI 上
    #   import 会失败。同时它靠 import 副作用把自己注册进本表，
    #   不 import 就查不到：所以在查表之前先尝试加载。
    if effective_backend in _OPTIONAL_DRIVER_MODULES:
        mod = _OPTIONAL_DRIVER_MODULES[effective_backend]
        if mod not in sys.modules:
            try:
                importlib.import_module(mod)
            except ImportError as exc:  # pragma: no cover - 取决于环境
                raise DriverError(
                    f"arm driver {effective_backend!r} requires module {mod!r}: {exc}"
                ) from exc
    vendors = driver_cfg.get("vendors")
    if isinstance(vendors, dict) and effective_backend in vendors:
        vendor = vendors[effective_backend]
        if not isinstance(vendor, dict):
            raise ValueError(f"vendor {effective_backend!r} must be a mapping")
        kind = str(vendor.get("kind", "http"))
        factory = ARM_DRIVER_REGISTRY.get(kind)
        if factory is None:
            raise ValueError(
```

```
                f"unknown driver kind {kind!r}; registered: "
                f"{sorted(ARM_DRIVER_REGISTRY)}"
            )
        params = {key: value for key, value in vendor.items() if key != "kind"}
        return factory(**params)
    factory = ARM_DRIVER_REGISTRY.get(effective_backend)
    if factory is None:
        raise ValueError(
            f"unknown arm driver backend {effective_backend!r}; registered: "
            f"{sorted(ARM_DRIVER_REGISTRY)}"
        )
    params = driver_cfg.get(effective_backend)
    if not isinstance(params, dict):
        params = {}
    return factory(**params)
class SimulatedArmDriver:
    """Deterministic arm driver for no-hardware self tests.
    It performs no physical action. It exists so the full
    task -> ack -> progress -> telemetry loop can be verified before a real
    arm arrives.
    """
    def __init__(
        self,
        battery: int = 100,
        bins: dict[str, float] | None = None,
        home: dict[str, float] | None = None,
        collected_weight: float = 0.05,
        review_result: str = "confirmed",
    ) -> None:
        self.battery = int(battery)
        self.bins: dict[str, float] = dict(
            bins or {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
        )
        home = home or {"lng": 119.6540, "lat": 26.3870}
        self.home = Position(float(home["lng"]), float(home["lat"]))
        self.mode = DeviceMode.IDLE
        self.collected_weight = float(collected_weight)
        self.review_result = review_result
        self.pick_calls = 0
    def status(self) -> ArmStatus:
        return ArmStatus(
            mode=self.mode,
            battery=self.battery,
            bins=dict(self.bins),
            location=self.home,
        )
    def pick(self, task_id: str, target: Position, priority: int) -> PickResult:
        self.pick_calls += 1
        self.mode = DeviceMode.IDLE
        self.bins["foam"] = min(
```

```
            1.0, self.bins["foam"] + self.collected_weight * 2.0
        )
        return PickResult(
            ok=True,
            collected_weight=self.collected_weight,
            review_result=self.review_result,
            bins_after=dict(self.bins),
        )
    def pause(self) -> None:
        self.mode = DeviceMode.PAUSED
    def resume(self) -> None:
        self.mode = DeviceMode.NAVIGATING if self.mode == DeviceMode.PAUSED else DeviceMode.IDLE
    def return_home(self) -> None:
        self.mode = DeviceMode.IDLE
    def emergency_stop(self) -> None:
        self.mode = DeviceMode.E_STOP
class HttpArmDriver:
    """Generic HTTP adapter example for arms that expose an HTTP SDK.
    Configure endpoint to the arm's control endpoint. Each call sends one
    JSON body with an action field; the response may carry optional
    mode / ok / collected_weight / review_result /
    evidence_url / bins_after fields.
    """
    def __init__(
        self,
        endpoint: str,
        method: str = "POST",
        timeout: float = 10.0,
        headers: dict[str, str] | None = None,
        battery: int = 100,
        bins: dict[str, float] | None = None,
        home: dict[str, float] | None = None,
    ) -> None:
        self.endpoint = str(endpoint).rstrip("/")
        self.method = str(method).upper()
        self.timeout = float(timeout)
        self.headers = {"Content-Type": "application/json"}
        self.headers.update(headers or {})
        self.battery = int(battery)
        self.bins: dict[str, float] = dict(
            bins or {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
        )
        home = home or {"lng": 119.6540, "lat": 26.3870}
        self.home = Position(float(home["lng"]), float(home["lat"]))
        self.mode = DeviceMode.IDLE
    def status(self) -> ArmStatus:
        return ArmStatus(
            mode=self.mode,
            battery=self.battery,
            bins=dict(self.bins),
```

```
            location=self.home,
        )
    def _call(self, action: str, payload: dict[str, Any]) -> dict[str, Any]:
        body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
        request = urllib.request.Request(
            self.endpoint, data=body, method=self.method, headers=self.headers
        )
        try:
            with urllib.request.urlopen(request, timeout=self.timeout) as response:
                raw = response.read()
        except urllib.error.HTTPError as exc:
            raise DriverError(f"{action} failed with HTTP {exc.code}") from exc
        except OSError as exc:
            raise DriverError(f"{action} failed: {exc}") from exc
        if not raw:
            return {}
        try:
            data = json.loads(raw.decode("utf-8"))
        except ValueError:
            return {}
        if not isinstance(data, dict):
            return {}
        self.mode = str(data.get("mode", self.mode))
        return data
    def pick(self, task_id: str, target: Position, priority: int) -> PickResult:
        data = self._call(
            "pick",
            {
                "action": "pick",
                "task_id": task_id,
                "target": target.to_dict(),
                "priority": int(priority),
            },
        )
        bins = data.get("bins_after")
        return PickResult(
            ok=bool(data.get("ok", True)),
            collected_weight=float(data.get("collected_weight", 0.0)),
            review_result=str(data.get("review_result", "confirmed")),
            evidence_url=data.get("evidence_url"),
            bins_after=dict(bins) if isinstance(bins, dict) else dict(self.bins),
        )
    def pause(self) -> None:
        self._call("pause", {"action": "pause"})
    def resume(self) -> None:
        self._call("resume", {"action": "resume"})
    def return_home(self) -> None:
        self._call("return_home", {"action": "return_home"})
    def emergency_stop(self) -> None:
        self._call("emergency_stop", {"action": "emergency_stop"})
```

```
class HiwonderBusServoArmDriver:


    """Adapter for Hiwonder Raspberry Pi bus-servo arms.
    The vendor ros_robot_controller_sdk.Board talks to the expansion board
    over the Pi UART (default /dev/ttyAMA0 at 1 Mbaud). The platform's
    GPS target is intentionally not treated as joint coordinates: pick
    plays a pre-taught pick_sequence (servo id / position pairs recorded
    with the vendor's teach-and-playback example), then returns the platform
    PickResult.
    The SDK is imported lazily so this module stays testable and importable on
    machines without pyserial or the arm attached. Physical actions raise
    DriverError when the SDK is unavailable; status still returns
    the configured deterministic snapshot for config self checks.
    """
    def __init__(
        self,
        serial_port: str = "/dev/ttyAMA0",
        baudrate: int = 1000000,
        timeout: float = 5.0,
        servo_ids: list[int] | None = None,
        home_positions: list[list[int]] | None = None,
        pick_sequence: list[dict[str, Any]] | None = None,
        pick_sequence_file: str | None = None,
        pick_positions: list[list[int]] | None = None,
        pick_duration: float = 1.0,
        home_duration: float = 1.5,
        collected_weight: float = 0.0,
        review_result: str = "recheck",
        evidence_url: str | None = None,
        battery: int = 100,
        bins: dict[str, float] | None = None,
        home: dict[str, float] | None = None,
    ) -> None:
        self.serial_port = str(serial_port)
        self.baudrate = int(baudrate)
        self.timeout = float(timeout)
        self.servo_ids = [int(sid) for sid in (servo_ids or [1, 2, 3, 4])]
        if not self.servo_ids:
            raise ValueError("servo_ids must not be empty")
        self.home_positions = self._normalize_positions(home_positions or [])
        if pick_sequence_file is not None and pick_sequence is not None:
            raise ValueError(
                "use either pick_sequence or pick_sequence_file, not both"
            )
        if pick_sequence_file is not None:
            self.pick_sequence = self._load_sequence_file(
                str(pick_sequence_file)
            )
        else:
            self.pick_sequence = self._normalize_sequence(pick_sequence)
        if not self.pick_sequence and pick_positions:
```

```
            self.pick_sequence = [
                {
                    "duration": float(pick_duration),
                    "positions": self._normalize_positions(pick_positions),
                }
            ]
        if not self.pick_sequence:
            raise ValueError(
                "pick_sequence or pick_positions is required for a physical arm"
            )
        self.pick_duration = float(pick_duration)
        self.home_duration = float(home_duration)
        self.collected_weight = float(collected_weight)
        self.review_result = str(review_result)
        self.evidence_url = evidence_url
        self.battery = int(battery)
        self.bins: dict[str, float] = dict(
            bins or {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
        )
        home = home or {"lng": 119.6540, "lat": 26.3870}
        self.home = Position(float(home["lng"]), float(home["lat"]))
        self._mode = DeviceMode.IDLE
        self._board: Any | None = None
    @staticmethod
    def _normalize_positions(positions: Any) -> list[list[int]]:
        normalized: list[list[int]] = []
        for item in positions:
            if not isinstance(item, (list, tuple)) or len(item) != 2:
                raise ValueError(
                    "servo positions must be [servo_id, position] pairs"
                )
            servo_id = int(item[0])
            position = int(item[1])
            if servo_id < 0:
                raise ValueError(f"servo_id must be >= 0, got {servo_id}")
            if not 0 <= position <= 1000:
                raise ValueError(
                    f"bus servo position must be 0..1000, got {position}"
                )
            normalized.append([servo_id, position])
        return normalized
    @classmethod
    def _normalize_sequence(cls, sequence: Any) -> list[dict[str, Any]]:
        normalized: list[dict[str, Any]] = []
        for step in sequence or []:
            if isinstance(step, dict):
                duration = float(step["duration"])
                positions = cls._normalize_positions(step["positions"])
            elif isinstance(step, (list, tuple)) and len(step) == 2:
                duration = float(step[0])
```

```
            positions = cls._normalize_positions(step[1])
        else:
            raise ValueError(
                "pick_sequence steps must be {duration, positions} maps "
                "or [duration, [[servo_id, position], ...]] pairs"
            )
        if duration <= 0:
            raise ValueError(f"pick step duration must be > 0, got {duration}")
        normalized.append({"duration": duration, "positions": positions})
    return normalized
@classmethod
def _load_sequence_file(cls, path: str) -> list[dict[str, Any]]:
    """Load a pick sequence JSON file written by a teach tool.
    Accepts three shapes, because the vendor tooling and ours differ:
    1. Vendor format — a flat list of single-servo positions::
           [1000, 940]
       That is what 案例5 示教记录实现/bus_servo_record.py writes:
       it records one position per Enter press, for ONE servo. Converted
       here into single-servo steps so a vendor-recorded file can be
       replayed without hand-editing.
    2. Our format — a list of {"duration":…, "positions": [[id,pos]]}.
    3. {"steps": [...]} wrapping either of the above.
    """
    try:
        raw = json.loads(Path(path).read_text(encoding="utf-8"))
    except OSError as exc:
        raise ValueError(
            f"cannot read pick_sequence_file {path!r}: {exc}"
        ) from exc
    except json.JSONDecodeError as exc:
        raise ValueError(
            f"pick_sequence_file {path!r} is not valid JSON: {exc}"
        ) from exc
    if isinstance(raw, dict) and isinstance(raw.get("steps"), list):
        raw = raw["steps"]
    if not isinstance(raw, list):
        raise ValueError(
            f"pick_sequence_file {path!r} must contain a list of steps "
            "or a {'steps': [...]} mapping"
        )
    # 厂商扁平格式 [1000, 940, …] → 归一化成 step 列表。
    #   厂商示教程序一次只记一个舵机的位置（用 bus_servo_read_id() 取
    #   第一个舵机），所以这里按配置的 servo_ids[0] 还原。
    if raw and all(isinstance(x, (int, float)) for x in raw):
        return cls._normalize_sequence(
            [
                {
                    "duration": 1.0,
                    "positions": [[1, int(round(float(x)))]],
                }
```

```
                for x in raw
            ]
        )
    return cls._normalize_sequence(raw)
def _new_board(self) -> Any:
    # 先查串口在不在，再 import SDK：顺序反过来会让现场拿到
    # "SDK unavailable" 这种指向错误的提示（真机没串口时 SDK 是有的，
    # 真正的原因是舵机走 ROS 而不是串口）。
    if not Path(self.serial_port).exists():
        import glob  # noqa: PLC0415
        tty_usb = sorted(glob.glob("/dev/ttyUSB*"))
        tty_acm = sorted(glob.glob("/dev/ttyACM*"))
        found = ", ".join(tty_usb + tty_acm) or "none"
        raise DriverError(
            f"serial port {self.serial_port} does not exist "
            f"(ttyUSB/ttyACM present: {found}). "
            "Either fix driver.serial_port in config.yaml, or — if the "
            "arm is driven over ROS instead of a UART — the platform "
            "needs a RosArmDriver implementing the same ArmDriver "
            "protocol; check `rosnode list` / `rostopic list` on the Pi. "
            "See docs/competitions/arm-hardware-probe-2026-10-05.md."
        )
    try:
        import ros_robot_controller_sdk as rrc  # noqa: PLC0415
    except ImportError as exc:
        raise DriverError(
            "ros_robot_controller_sdk is unavailable; copy the vendor "
            "ros_robot_controller_sdk.py onto the Pi next to the bridge "
            "and install pyserial"
        ) from exc
    try:
        return rrc.Board(
            device=self.serial_port,
            baudrate=self.baudrate,
            timeout=self.timeout,
        )
    except Exception as exc:  # noqa: BLE001
        raise DriverError(
            f"cannot open Hiwonder board on {self.serial_port}: {exc}"
        ) from exc
def _get_board(self) -> Any:
    if self._board is None:
        self._board = self._new_board()
    return self._board
def _stop_and_release(self, board: Any) -> None:
    board.bus_servo_stop(self.servo_ids)
    for servo_id in self.servo_ids:
        board.bus_servo_enable_torque(servo_id, False)
def _read_servo_telemetry(self, board: Any) -> dict[str, Any]:
    """逐个舵机回读电压/温度/位置。
```

```
        这是"机械臂真的动了"的硬证据：progress 报文是平台自报，


        而这里的数字来自舵机本身。答辩时被问"怎么证明真机执行了"，
        拿这个出来比任何架构图都有说服力。
        单个舵机读失败不影响其余（每个都独立 try），因为总线的
        回读是异步队列、单次调用常常正好赶不上回包。
        """
        out: dict[str, Any] = {}
        for sid in self.servo_ids:
            item: dict[str, Any] = {}
            for key, fn in (
                ("vin", board.bus_servo_read_vin),
                ("temp", board.bus_servo_read_temp),
                ("position", board.bus_servo_read_position),
            ):
                try:
                    val = fn(sid)
                    # 厂商 SDK 返回 list，取第一个元素
                    if isinstance(val, (list, tuple)):
                        val = val[0] if val else None
                    if isinstance(val, (int, float)):
                        item[key] = round(float(val), 2)
                except Exception:  # noqa: BLE001
                    pass
            if item:
                out[str(sid)] = item
        return out
    def _read_board_voltage(self, board: Any, tries: int = 6) -> float | None:
        """读控制板电压（mV）。
        `Board.get_battery()` 是非阻塞的：它只从已收到的队列里取，
          队列空就直接返回 None。所以必须先 enable_reception、稍等、
          再重试几次：原实现只调一次，等于永远读不到，battery 恒为
          构造时的初值。这是"声明了却没实现"的静默缺陷。
        """
        for _ in range(tries):
            try:
                raw = board.get_battery()
            except Exception:  # noqa: BLE001
                return None
            if isinstance(raw, (int, float)) and raw > 0:
                return float(raw)
            time.sleep(0.05)
        return None
    def status(self) -> ArmStatus:
        battery = self.battery
        servos: dict[str, Any] = {}
        board = self._board
        if board is not None:
            try:
                board.enable_reception(True)
                 time.sleep(0.1)
```

```
            servos = self._read_servo_telemetry(board)



            mv = self._read_board_voltage(board)
            if mv is not None:
                # 控制板电压来自 2S 锂电（标称 7.4V，满电约 8.4V）。
                # 这里是粗略换算，仅用于"电量明显偏低"的告警，
                # 不作为精确 SoC：精确 SoC 需要放电曲线标定，暂无。
                battery = max(0, min(100, round((mv - 6000.0) / 2400.0 * 100)))
        except Exception:  # noqa: BLE001
            # 读不到就保持上次的值；不能因遥测失败让整条链路挂掉
            pass
    return ArmStatus(
        mode=self._mode,
        battery=battery,
        bins=dict(self.bins),
        location=self.home,
        servos=servos,
    )
def pick(self, task_id: str, target: Position, priority: int) -> PickResult:
    board = self._get_board()
    self._mode = DeviceMode.COLLECTING
    try:
        board.enable_reception(True)
        for servo_id in self.servo_ids:
            board.bus_servo_enable_torque(servo_id, True)
        for step in self.pick_sequence:
            board.bus_servo_set_position(
                float(step["duration"]), step["positions"]
            )
            time.sleep(max(0.0, float(step["duration"]) + 0.2))
        board.bus_servo_stop(self.servo_ids)
    except DriverError:
        self._mode = DeviceMode.IDLE
        raise
    except Exception as exc:  # noqa: BLE001
        self._mode = DeviceMode.IDLE
        raise DriverError(f"Hiwonder pick failed: {exc}") from exc
    self.bins["foam"] = min(1.0, self.bins["foam"] + self.collected_weight)
    self._mode = DeviceMode.IDLE
    return PickResult(
        ok=True,
        collected_weight=self.collected_weight,
        review_result=self.review_result,
        evidence_url=self.evidence_url,
        bins_after=dict(self.bins),
    )
def pause(self) -> None:
    board = self._get_board()
    try:
        board.bus_servo_stop(self.servo_ids)
    except Exception as exc:  # noqa: BLE001
```

```
            raise DriverError(f"Hiwonder pause failed: {exc}") from exc



        self._mode = DeviceMode.PAUSED
    def resume(self) -> None:
        self._mode = (
            DeviceMode.NAVIGATING
            if self._mode == DeviceMode.PAUSED
            else DeviceMode.IDLE
        )
    def return_home(self) -> None:
        board = self._get_board()
        try:
            if self.home_positions:
                board.bus_servo_set_position(
                    self.home_duration, self.home_positions
                )
                time.sleep(max(0.0, self.home_duration + 0.2))
            board.bus_servo_stop(self.servo_ids)
        except Exception as exc:  # noqa: BLE001
            raise DriverError(f"Hiwonder return_home failed: {exc}") from exc
        self._mode = DeviceMode.IDLE
    def emergency_stop(self) -> None:
        board = self._get_board()
        try:
            self._stop_and_release(board)
        except Exception as exc:  # noqa: BLE001
            raise DriverError(f"Hiwonder emergency_stop failed: {exc}") from exc
        self._mode = DeviceMode.E_STOP
register_arm_driver("simulated", SimulatedArmDriver)
register_arm_driver("http", HttpArmDriver)
register_arm_driver("hiwonder_bus_servo", HiwonderBusServoArmDriver)
# ===== edge/arm_bridge/ros_driver.py =====
"""通过厂商 ROS 控制栈驱动机械臂（ros_control action）。
## 为什么需要这个驱动
2026-10-05 实测发现：厂商的 hiwonder_servo_manager 节点已经持有
/dev/ttyS0，并且 /joint_states 在正常发布 6 个关节的实时角度。
也就是说机械臂已被厂商 ROS 栈接管，我们的驱动若再去直连串口：
* 物理上抢不到总线（半双工，一发一收必须独占）
* 逻辑上也不该抢：绕过厂商程序会失去它的逆运动学与动作组
所以这里实现的是同一个 `ArmDriver` 协议的另一种执行端：
平台与桥接层一行不改，只把 driver.backend 换成 ros_arm_control。
这正是"执行端可替换层"最实在的例证：驱动的实现从"直连串口"
换成"ROS action"，平台侧的派单逻辑、状态机、证据链完全不动。
## 与 HiwonderBusServoArmDriver 的差别
=====================  ========================  ==========================
                       HiwonderBusServo（串口）   RosArmDriver（ROS action）
=====================  ========================  ==========================
控制方式                示教位置序列直接下发      关节角轨迹 /末端坐标
位置语义                脉宽 0–1000（无量纲）     弧度（有物理意义）
机械爪                  序列里的一组舵机          /gripper_controller 独立
遥测                    读电压/温度/脉宽          读 /joint_states 角度
```

占用关系　　与厂商程序互斥与厂商程序共存

========== ========== ==========

串口那条路必须先停掉厂商节点才能用，ROS 这条路不用。

"""

```
from __future__ import annotations
import math
import time
from dataclasses import dataclass, field
from typing import Any
# 用相对导入：bridge.py 以 `from arm_bridge.drivers import ...` 方式加载，
#   若这里写 `from drivers import`，Python 会把它当成另一个顶层模块，
#   于是注册进的是两份不同的 ARM_DRIVER_REGISTRY，配置选 ros_arm_control
#   会报 KeyError。同理 device_sim 也要用完整包路径。
from .drivers import (
    ArmDriver,
    ArmStatus,
    DeviceMode,
    PickResult,
    Position,
    register_arm_driver,
)
# 关节名 → 我们的 servo_id 映射。厂商 URDF 用 joint1..5 + r_joint。
# 这是读回用的（遥测展示）；下发轨迹时顺序必须与 URDF 一致，
#   所以两者共用这一个列表，避免两处漂移。
DEFAULT_JOINT_NAMES = [
    "joint1",
    "joint2",
    "joint3",
    "joint4",
    "joint5",
    "r_joint",
]
# home 位（弧度）—— 与实测 /joint_states 的厂商标定值一致。
DEFAULT_HOME_RAD = [0.0, 0.5236, -1.3614, -1.7593, 0.0, -1.7802]
# 抓取姿态（弧度）。这是默认值，真机首跑前应当示教确认——
#   直接用 home 位当抓取位会抓空。配置里可覆盖。
DEFAULT_PICK_RAD = [0.0, 0.5236, -1.3614, -1.7593, 0.0, -1.7802]
@dataclass
class _JointSnapshot:
    """一次 /joint_states 读数。"""
    names: list[str] = field(default_factory=list)
    positions: list[float] = field(default_factory=list)
    velocities: list[float] = field(default_factory=list)
    efforts: list[float] = field(default_factory=list)
    stamp: float = 0.0
    def as_servo_map(self) -> dict[str, Any]:
        """转成 ArmStatus.servos 用的 {id: {...}} 结构。
        键用舵机序号（1..n）而不是 joint 名，与 HiwonderBusServo 那条
        路径的键类型保持一致：页面的展示逻辑不必为两种驱动写两套。
        """
```

```
        out: dict[str, Any] = {}

        for i, pos in enumerate(self.positions, start=1):
            item: dict[str, Any] = {
                # 弧度 + 度，度更直观；position 保留原值便于核对
                "position": round(pos, 4),
                "position_deg": round(math.degrees(pos), 2),
                "joint": self.names[i - 1] if i - 1 < len(self.names) else "?",
            }
            if i - 1 < len(self.velocities):
                item["velocity"] = round(self.velocities[i - 1], 4)
            if i - 1 < len(self.efforts):
                item["effort"] = round(self.efforts[i - 1], 3)
            out[str(i)] = item
        return out
class RosArmDriver:
    """用 ros_control action 驱动厂商机械臂，实现同一份 ArmDriver 协议。
    ROS 依赖是惰性导入的：没装 rospy 的机器（比如开发机、CI）仍能
    import 本模块并跑其余驱动的测试，只是不注册 ros_arm_control——
    这与 HiwonderBusServoArmDriver 对 pyserial /厂商 SDK 的处理一致。
    """
    def __init__(
        self,
        arm_controller: str = "arm_controller",
        gripper_controller: str = "gripper_controller",
        joint_states_topic: str = "/joint_states",
        joint_names: list[str] | None = None,
        home_rad: list[float] | None = None,
        pick_rad: list[float] | None = None,
        move_timeout: float = 15.0,
        action_done_timeout: float = 20.0,
        collected_weight: float = 0.0,
        review_result: str = "recheck",
        evidence_url: str | None = None,
        battery: int = 100,
        bins: dict[str, float] | None = None,
        home: dict[str, float] | None = None,
    ) -> None:
        self.arm_controller = str(arm_controller)
        self.gripper_controller = str(gripper_controller)
        self.joint_states_topic = str(joint_states_topic)
        self.joint_names = list(joint_names or DEFAULT_JOINT_NAMES)
        self.home_rad = [float(x) for x in (home_rad or DEFAULT_HOME_RAD)]
        self.pick_rad = [float(x) for x in (pick_rad or DEFAULT_PICK_RAD)]
        self.move_timeout = float(move_timeout)
        self.action_done_timeout = float(action_done_timeout)
        self.collected_weight = float(collected_weight)
        self.review_result = str(review_result)
        self.evidence_url = evidence_url
        self.battery = int(battery)
        self.bins: dict[str, float] = dict(
```

```
            bins or {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
        )
        self.home: Position = Position(
            float((home or {}).get("lng", 119.6540)),
            float((home or {}).get("lat", 26.3870)),
        )
        self._mode = DeviceMode.IDLE
        self._node: Any = None
        self._last: _JointSnapshot | None = None
    # ROS 惰性接入
    def _ensure_node(self) -> Any:
        """建 rospy 节点并订阅 /joint_states（只订阅，不发任何指令）。"""
        if self._node is not None:
            return self._node
        try:
            import rospy
            from sensor_msgs.msg import JointState
        except ImportError as exc:  # pragma: no cover - 取决于运行环境
            raise RuntimeError(
                "ROS Python (rospy + sensor_msgs) is unavailable. Install "
                "ros-<distro>-rospy on the arm host, or pick another "
                "driver.backend (e.g. simulated / http)."
            ) from exc
        rospy.init_node(
            "seasight_ros_arm_driver", anonymous=True, disable_signals=True
        )
        self._node = rospy
        self._JointState = JointState
        self._node.Subscriber(
            self.joint_states_topic, JointState, self._on_joint_states, queue_size=1
        )
        # 给订阅一点时间拿到第一帧，否则首次 status() 会是空
        deadline = time.time() + 3.0
        while time.time() < deadline and self._last is None:
            time.sleep(0.1)
        return self._node
    def _on_joint_states(self, msg: Any) -> None:  # pragma: no cover - 需 ROS
        self._last = _JointSnapshot(
            names=list(msg.name),
            positions=[float(p) for p in msg.position],
            velocities=[float(v) for v in msg.velocity],
            efforts=[float(e) for e in msg.effort],
            stamp=msg.header.stamp.to_sec() if msg.header else 0.0,
        )
    # ArmDriver 协议
    def status(self) -> ArmStatus:
        """读当前关节角作为遥测。
        读不到时返回上次快照（可能为空 dict）而不抛异常：遥测失败不该
        让整条平台链路挂掉，这一点与串口驱动的处理一致。
        """
```

```
        servos: dict[str, Any] = {}



        try:
            self._ensure_node()
            if self._last is not None:
                servos = self._last.as_servo_map()
        except Exception:  # noqa: BLE001
            pass
        return ArmStatus(
            mode=self._mode,
            battery=self.battery,
            bins=dict(self.bins),
            location=self.home,
            servos=servos,
        )
    def pick(self, task_id: str, target: Position, priority: int) -> PickResult:
        """执行一次抓取。
        重要口径：target 是平台的 GPS 目标点，不是关节角。
          末端坐标 → 关节角的逆运动学由厂商 ROS 栈负责
          （armpi_fpv_kinematics），我们不自己解：解了也对不上
          厂商的标定。逆运动学源码未取回前，不要宣称"坐标级抓取精度"。
        """
        rospy = self._ensure_node()
        self._mode = DeviceMode.COLLECTING
        try:
            self._send_trajectory(self.pick_rad)
            self._close_gripper(rospy)
            self._open_gripper(rospy)
        except Exception as exc:  # noqa: BLE001
            self._mode = DeviceMode.IDLE
            raise RuntimeError("ROS pick failed: %s" % exc) from exc
        self._mode = DeviceMode.IDLE
        self.bins["foam"] = min(
            1.0, self.bins["foam"] + self.collected_weight
        )
        return PickResult(
            ok=True,
            collected_weight=self.collected_weight,
            review_result=self.review_result,
            evidence_url=self.evidence_url,
            bins_after=dict(self.bins),
        )
    def _send_trajectory(self, positions: list[float]) -> None:  # pragma: no cover
        """经 follow_joint_trajectory action 下发关节角轨迹。"""
        from control_msgs.msg import FollowJointTrajectoryGoal
        from trajectory_msgs.msg import JointTrajectoryPoint
        rospy = self._node
        goal = FollowJointTrajectoryGoal()
        goal.trajectory.joint_names = list(self.joint_names)
        point = JointTrajectoryPoint()
        point.positions = [float(x) for x in positions]
```

```
        point.time_from_start = rospy.Duration(self.move_timeout)



        goal.trajectory.points = [point]
        client = rospy.SimpleActionClient(
            "/%s/follow_joint_trajectory" % self.arm_controller,
            FollowJointTrajectoryGoal,
        )
        if not client.wait_for_server(rospy.Duration(5.0)):
            raise RuntimeError(
                "action server /%s/follow_joint_trajectory not available"
                % self.arm_controller
            )
        client.send_goal(goal)
        if not client.wait_for_result(rospy.Duration(self.action_done_timeout)):
            client.cancel_goal()
            raise RuntimeError("trajectory action timed out")
        state = client.get_state()
        label = {
            3: "succeeded",
            4: "aborted",
            5: "preempted",
        }.get(state, "state=%s" % state)
        if label != "succeeded":
            raise RuntimeError("trajectory %s" % label)
    def _close_gripper(self, rospy: Any) -> None:  # pragma: no cover
        """收拢机械爪（gripper_controller 独立于 arm_controller）。"""
        from control_msgs.msg import GripperCommand
        pub = rospy.Publisher(
            "/%s/command" % self.gripper_controller, GripperCommand, queue_size=1
        )
        for _ in range(10):
            if pub.get_num_connections() > 0:
                break
            rospy.sleep(0.2)
        pub.publish(GripperCommand(position=0.0, max_effort=1.0))
        rospy.sleep(1.0)
    def _open_gripper(self, rospy: Any) -> None:  # pragma: no cover
        from control_msgs.msg import GripperCommand
        pub = rospy.Publisher(
            "/%s/command" % self.gripper_controller, GripperCommand, queue_size=1
        )
        for _ in range(10):
            if pub.get_num_connections() > 0:
                break
            rospy.sleep(0.2)
        pub.publish(GripperCommand(position=1.0, max_effort=1.0))
        rospy.sleep(1.0)
    def pause(self) -> None:
        self._mode = DeviceMode.PAUSED
    def resume(self) -> None:
        self._mode = (
```

```
            DeviceMode.NAVIGATING if self._mode == DeviceMode.PAUSED
            else DeviceMode.IDLE
        )
    def return_home(self) -> None:  # pragma: no cover
        try:
            rospy = self._ensure_node()
            self._send_trajectory(self.home_rad)
        except Exception:  # noqa: BLE001
            pass
        self._mode = DeviceMode.IDLE
    def emergency_stop(self) -> None:  # pragma: no cover
        """急停：取消所有正在执行的轨迹，并置 E_STOP。
        取消而不是"发 home"：急停时最不该做的就是让机械臂继续动。
        """
        try:
            rospy = self._ensure_node()
            from control_msgs.msg import FollowJointTrajectoryGoal
            client = rospy.SimpleActionClient(
                "/%s/follow_joint_trajectory" % self.arm_controller,
                FollowJointTrajectoryGoal,
            )
            if client.wait_for_server(rospy.Duration(2.0)):
                client.cancel_all_goals()
                client.cancel_goal()
        except Exception:  # noqa: BLE001
            pass
        self._mode = DeviceMode.E_STOP
# 惰性注册：只有 import 本模块才会进注册表，避免污染其它驱动的测试环境。
register_arm_driver("ros_arm_control", RosArmDriver)
__all__ = ["RosArmDriver", "DEFAULT_JOINT_NAMES", "DEFAULT_HOME_RAD"]
# ===== edge/arm_bridge/mqtt_transport.py =====
"""Real MQTT transport for the arm bridge.
This is a thin adapter over paho-mqtt that implements the same transport
contract used by tests, so the bridge itself stays transport-agnostic.
"""
from __future__ import annotations
import json
import logging
from typing import Any
logger = logging.getLogger("arm_bridge.mqtt")
class MqttTransport:
    """paho-mqtt based transport with LWT and exact-topic routing."""
    def __init__(
        self,
        host: str = "localhost",
        port: int = 1883,
        username: str | None = None,
        password: str | None = None,
        client_id: str = "robot-arm-bridge",
        keepalive: int = 60,
```

```
        use_lwt: bool = True,
        will_topic: str | None = None,
    ) -> None:
        self.host = str(host)
        self.port = int(port)
        self.username = username
        self.password = password
        self.client_id = str(client_id)
        self.keepalive = int(keepalive)
        self.use_lwt = bool(use_lwt)
        self.will_topic = will_topic
        self.connected = False
        self._client: Any = None
        self._handlers: dict[str, list[Any]] = {}
    def connect(self) -> None:
        import paho.mqtt.client as mqtt  # noqa: PLC0415
        try:
            client = mqtt.Client(
                mqtt.CallbackAPIVersion.VERSION2,
                client_id=self.client_id,
                protocol=mqtt.MQTTv5,
            )
        except (AttributeError, ValueError):
            client = mqtt.Client(client_id=self.client_id)
        if self.username:
            client.username_pw_set(self.username, self.password or "")
        if self.use_lwt and self.will_topic:
            client.will_set(
                self.will_topic,
                json.dumps({"online": False}, ensure_ascii=False),
                qos=1,
                retain=True,
            )
        client.on_connect = self._on_connect
        client.on_disconnect = self._on_disconnect
        client.on_message = self._on_message
        client.connect(self.host, self.port, keepalive=self.keepalive)
        client.loop_start()
        self._client = client
    def disconnect(self) -> None:
        if self._client is None:
            return
        try:
            self._client.loop_stop()
            self._client.disconnect()
        except Exception:  # noqa: BLE001
            logger.warning("MQTT disconnect failed", exc_info=True)
        self._client = None
        self.connected = False
    def subscribe(self, topic: str, handler: Any) -> None:
```

```
        self._handlers.setdefault(topic, []).append(handler)



        if self.connected and self._client is not None:
            self._client.subscribe(topic, qos=1)
    def unsubscribe(self, topic: str, handler: Any) -> None:
        handlers = self._handlers.get(topic)
        if not handlers:
            return
        try:
            handlers.remove(handler)
        except ValueError:
            pass
        if not handlers:
            self._handlers.pop(topic, None)
            if self.connected and self._client is not None:
                self._client.unsubscribe(topic)
    def publish(self, topic: str, payload: dict[str, Any], qos: int = 1) -> bool:
        if not self.connected or self._client is None:
            return False
        info = self._client.publish(
            topic, json.dumps(payload, ensure_ascii=False), qos=qos, retain=False
        )
        return getattr(info, "rc", 0) == 0
    def _on_connect(self, client: Any, userdata: Any, flags: Any, reason_code: Any, properties: Any…
        self.connected = True
        for topic, handlers in self._handlers.items():
            client.subscribe(topic, qos=1)
        logger.info("arm bridge connected: %s", self.host)
    def _on_disconnect(self, *args: Any) -> None:
        self.connected = False
        logger.warning("arm bridge disconnected from %s", self.host)
    def _on_message(self, client: Any, userdata: Any, message: Any) -> None:
        topic = str(message.topic)
        try:
            payload = json.loads(message.payload) if message.payload else {}
        except ValueError:
            logger.warning("invalid JSON on %s", topic)
            return
        for handler in list(self._handlers.get(topic, ())):
            handler(topic, payload)
# ===== edge/arm_bridge/tools/teach_hiwonder_sequence.py =====
#!/usr/bin/env python3
"""Record a Hiwonder bus-servo arm pick sequence from physical poses.
Run this on the Raspberry Pi with the vendor ros_robot_controller_sdk.py
next to it. Move the arm to each pose, press Enter to record that pose, then
press q to save. The output JSON is accepted by
HiwonderBusServoArmDriver through pick_sequence_file.
"""
from __future__ import annotations
import argparse
import json
```

```
import sys



from pathlib import Path
from typing import Any
def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--device", default="/dev/ttyAMA0")
    parser.add_argument("--baudrate", type=int, default=1000000)
    parser.add_argument("--timeout", type=float, default=5.0)
    parser.add_argument("--servo-ids", type=int, nargs="+", required=True)
    parser.add_argument("--duration", type=float, default=1.0)
    parser.add_argument("--output", default="pick_sequence.json")
    return parser.parse_args(argv)
def read_positions(board: Any, servo_ids: list[int]) -> list[list[int]]:
    """Read current 0..1000 positions for every servo id."""
    positions: list[list[int]] = []
    for servo_id in servo_ids:
        value = board.bus_servo_read_position(servo_id)
        if not value:
            raise RuntimeError(
                f"servo {servo_id} returned no position; check wiring and "
                "enable_reception"
            )
        position = int(value[0])
        if not 0 <= position <= 1000:
            raise RuntimeError(
                f"servo {servo_id} reported position {position}, expected 0..1000"
            )
        positions.append([servo_id, position])
    return positions
def build_output(
    servo_ids: list[int],
    steps: list[dict[str, Any]],
    duration: float,
) -> dict[str, Any]:
    """Build the JSON document consumed by the driver's sequence loader."""
    return {
        "format": "seasight_hiwonder_pick_sequence",
        "version": 1,
        "servo_ids": list(servo_ids),
        "default_duration": float(duration),
        "steps": steps,
    }
def record_loop(
    board: Any,
    servo_ids: list[int],
    duration: float,
    output: str,
) -> int:
    steps: list[dict[str, Any]] = []
    print("Move the arm to each pose, then press Enter to record it.")
```

```
    print("Press q to save and exit.")



    while True:
        try:
            user_input = input(
                "Enter to record current pose, q to save and exit: "
            ).strip().lower()
        except (EOFError, KeyboardInterrupt):
            break
        if user_input == "q":
            break
        positions = read_positions(board, servo_ids)
        steps.append({"duration": float(duration), "positions": positions})
        print(f"recorded step {len(steps)}: {positions}")
    if not steps:
        print("no steps recorded; nothing written", file=sys.stderr)
        return 1
    payload = build_output(servo_ids, steps, duration)
    Path(output).write_text(
        json.dumps(payload, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    print(f"saved {len(steps)} steps to {output}")
    return 0
def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    if args.duration <= 0:
        print("--duration must be > 0", file=sys.stderr)
        return 2
    try:
        import ros_robot_controller_sdk as rrc  # noqa: PLC0415
    except ImportError as exc:
        print(
            "vendor SDK not found; copy ros_robot_controller_sdk.py next to "
            "this script and install pyserial",
            file=sys.stderr,
        )
        return 2
    board = rrc.Board(
        device=args.device,
        baudrate=args.baudrate,
        timeout=args.timeout,
    )
    board.enable_reception(True)
    try:
        return record_loop(board, args.servo_ids, args.duration, args.output)
    except Exception as exc:  # noqa: BLE001
        print(f"recording failed: {exc}", file=sys.stderr)
        return 2
    finally:
        try:
```

```
            board.bus_servo_stop(args.servo_ids)


            for servo_id in args.servo_ids:
                board.bus_servo_enable_torque(servo_id, False)
        except Exception:  # noqa: BLE001
            pass
if __name__ == "__main__":
    raise SystemExit(main())
# ===== edge/arm_bridge/config.yaml =====
# ============================================================
# Oceanus arm bridge configuration
# ============================================================
# The bridge subscribes to robot/{device_id}/task and
# robot/{device_id}/cmd, then translates arm driver results into
# ACK / progress / telemetry messages.
site_id: "lianjiang"
device_id: "RBT-ARM-01"
mqtt:
  host: "localhost"
  port: 1883
  username: "robot_device"
  password: "CHANGE_ME"
  keepalive: 60
  use_lwt: true
driver:
  # simulated = deterministic self test without hardware
  # http      = generic HTTP adapter example for an arm HTTP SDK
  backend: "simulated"
  simulated:
    battery: 100
    bins:
      foam: 0.0
      plastic: 0.0
      mixed: 0.0
    home:
      lng: 119.6540
      lat: 26.3870
    collected_weight: 0.05
    review_result: "confirmed"
  http:
    endpoint: "http://192.168.1.100:8080/arm"
    method: "POST"
    timeout: 10.0
    headers: {}
    battery: 100
    bins:
      foam: 0.0
      plastic: 0.0
      mixed: 0.0
    home:
      lng: 119.6540
```

```
    lat: 26.3870



# New physical arms follow the same shape. Register the driver class in
# drivers.py, then select it here by name:
#
#   backend: "dobot_example"
#   dobot_example:
#     endpoint: "http://192.168.1.101:8080/dobot"
#     ...
#
# ---- 真机：走厂商 ROS 控制栈（推荐，2026-10-05 实测可用）----
#
# 厂商的 hiwonder_servo_manager 节点已持有 /dev/ttyS0，直连串口会与它
# 抢总线（半双工，必须独占）。本驱动复用 ros_control action，
# 与厂商程序和平共存，还能拿到有物理意义的关节角（弧度）。
#
# 前提：bridge 与机械臂在同一台树莓派上（或至少能访问其 ROS master）。
# 实测 joint_states：[joint1..joint5, r_joint]
#   positions = [0.0, 0.5236, -1.3614, -1.7593, 0.0, -1.7802] rad
#             = [0°, 30°, -78°, -100.8°, 0°, -102°]  ← 厂商标定的 home 位
#
# ros_arm_control:
#   arm_controller: "arm_controller"
#   gripper_controller: "gripper_controller"
#   # home_rad 为实测厂商标定值；pick_rad 必须示教确认，
#   # 直接用 home 位当抓取位会抓空。
#   home_rad: [0.0, 0.5236, -1.3614, -1.7593, 0.0, -1.7802]
#   pick_rad: [0.0, 0.5236, -1.3614, -1.7593, 0.0, -1.7802]
# Hiwonder Raspberry Pi bus-servo arms use the vendor SDK on the Pi.
# 直连串口必须先停掉厂商节点（它占着 /dev/ttyS0），否则抢不到总线。
#   sudo systemctl stop armpi_fpv_bringup
# 优先级：能用 ros_arm_control 就别用这条：串口那条与厂商 GUI 互斥。
# pick_sequence positions are pre-taught servo positions (0..1000), not GPS
# coordinates. Record them with the vendor's teach-and-playback example.
# Alternatively record with tools/teach_hiwonder_sequence.py and set
# pick_sequence_file instead of the inline list below.
#
# Or keep vendor blocks separate with an explicit kind:
vendors:
  dobot_example:
    kind: "http"
    endpoint: "http://192.168.1.101:8080/dobot"
    method: "POST"
    timeout: 10.0
    headers: {}
    battery: 100
    bins:
      foam: 0.0
      plastic: 0.0
      mixed: 0.0
    home:
```

```
      lng: 119.6540
      lat: 26.3870
  hiwonder_bus_servo:
    kind: "hiwonder_bus_servo"
    serial_port: "/dev/ttyAMA0"
    baudrate: 1000000
    timeout: 5.0
    servo_ids: [1, 2, 3, 4]
    home_positions: [[1, 500], [2, 500]]
    # pick_sequence_file: "pick_sequence.json"
    pick_sequence:
      - duration: 1.0
        positions: [[1, 650], [2, 500]]
      - duration: 1.0
        positions: [[1, 300], [2, 600]]
    collected_weight: 0.0
    review_result: "recheck"
    battery: 100
    bins:
      foam: 0.0
      plastic: 0.0
      mixed: 0.0
    home:
      lng: 119.6540
      lat: 26.3870
# ===== edge/arm_bridge/test_bridge.py =====
"""Arm bridge tests using the in-memory transport (E1/E2).
No broker, no hardware, no network. The tests prove the MQTT contract loop:
task -> ACK -> collecting -> driver pick -> done -> marine telemetry.
"""
from __future__ import annotations
import json
import sys
from pathlib import Path
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
if str(EDGE) not in sys.path:
    sys.path.insert(0, str(EDGE))
from arm_bridge.bridge import ArmBridge  # noqa: E402
from arm_bridge.drivers import (  # noqa: E402
    ARM_DRIVER_REGISTRY,
    DriverError,
    HiwonderBusServoArmDriver,
    HttpArmDriver,
    SimulatedArmDriver,
    build_arm_driver,
)
from device_sim.protocol import (  # noqa: E402
    DeviceCommand,
    DeviceMode,
```

```
    FakeClock,
    Position,
    SequentialIdFactory,
)
from device_sim.transport import MemoryTransport  # noqa: E402
DEVICE = "RBT-ARM-01"
SITE = "lianjiang"
def make_command(
    command_id: str,
    action: str,
    seq: int = 1,
    *,
    issued_at: float = 0.0,
    ttl: float = 30.0,
    **params: object,
) -> dict:
    return {
        "command_id": command_id,
        "device_id": DEVICE,
        "seq": seq,
        "action": action,
        "issued_at": issued_at,
        "expires_at": issued_at + ttl,
        "params": params,
    }
def dispatch_payload(
    task_id: str = "tsk_0001", issued_at: float = 0.0, ttl: float = 30.0
) -> dict:
    return make_command(
        f"cmd_{task_id}",
        "dispatch",
        task_id=task_id,
        target={"lng": 119.6531, "lat": 26.3867},
        priority=1,
        issued_at=issued_at,
        ttl=ttl,
    )
def make_bridge(start: float = 0.0) -> tuple[ArmBridge, MemoryTransport, FakeClock]:
    transport = MemoryTransport()
    clock = FakeClock(start)
    driver = SimulatedArmDriver()
    bridge = ArmBridge(
        DEVICE,
        SITE,
        driver,
        transport,
        clock=clock,
        id_factory=SequentialIdFactory("ack"),
    )
    bridge.start()
```

```
    return bridge, transport, clock


def published_on(transport: MemoryTransport, topic: str) -> list[dict]:
    return [payload for t, payload, qos in transport.published if t == topic]
def test_full_dispatch_loop() -> None:
    bridge, transport, clock = make_bridge()
    payload = dispatch_payload("tsk_0001")
    transport.deliver(bridge.task_topic, payload, qos=1)
    acks = published_on(transport, bridge.ack_topic)
    assert len(acks) == 1
    assert acks[0]["command_id"] == "cmd_tsk_0001"
    assert acks[0]["accepted"] is True
    assert acks[0]["mode"] == DeviceMode.NAVIGATING
    progress = published_on(transport, bridge.progress_topic)
    assert [p["status"] for p in progress] == ["collecting", "done"]
    assert progress[-1]["task_id"] == "tsk_0001"
    assert progress[-1]["robot_id"] == DEVICE
    assert progress[-1]["review_result"] == "confirmed"
    assert progress[-1]["collected_weight"] == 0.05
    telemetry = published_on(transport, bridge.telemetry_topic)
    assert len(telemetry) >= 2
    assert telemetry[0]["status"] == DeviceMode.COLLECTING
    assert telemetry[-1]["status"] == DeviceMode.IDLE
    assert telemetry[-1]["device_type"] == "robot"
    assert telemetry[-1]["task_id"] == "tsk_0001"
    assert bridge.counter["pick_executed"] == 1
def test_duplicate_command_is_idempotent() -> None:
    bridge, transport, _ = make_bridge()
    payload = dispatch_payload("tsk_0002")
    transport.deliver(bridge.task_topic, payload, qos=1)
    transport.deliver(bridge.task_topic, payload, qos=1)
    assert bridge.counter["duplicates"] == 1
    assert bridge.counter["pick_executed"] == 1
    assert len(published_on(transport, bridge.progress_topic)) == 2  # one loop only
    assert len(published_on(transport, bridge.ack_topic)) == 2  # original + replay
def test_expired_command_rejected() -> None:
    bridge, transport, clock = make_bridge()
    clock.advance(10.0)
    payload = dispatch_payload("tsk_0003", issued_at=0.0, ttl=5.0)
    transport.deliver(bridge.task_topic, payload, qos=1)
    acks = published_on(transport, bridge.ack_topic)
    assert len(acks) == 1
    assert acks[0]["accepted"] is False
    assert acks[0]["reason"] == "expired"
    assert bridge.counter["commands_rejected"] == 1
    assert bridge.counter["pick_executed"] == 0
    assert published_on(transport, bridge.progress_topic) == []
def test_missing_target_rejected() -> None:
    bridge, transport, _ = make_bridge()
    payload = make_command("cmd_tsk_0004", "dispatch", task_id="tsk_0004")
    transport.deliver(bridge.task_topic, payload, qos=1)
```

```
    acks = published_on(transport, bridge.ack_topic)



    assert acks[0]["accepted"] is False
    assert acks[0]["reason"] == "missing_target"
    assert bridge.counter["pick_executed"] == 0
def test_unknown_action_rejected() -> None:
    bridge, transport, _ = make_bridge()
    payload = make_command("cmd_0005", "fly")
    transport.deliver(bridge.cmd_topic, payload, qos=1)
    acks = published_on(transport, bridge.ack_topic)
    assert acks[0]["accepted"] is False
    assert acks[0]["reason"] == "unknown_action"
def test_non_dispatch_command_calls_driver() -> None:
    bridge, transport, _ = make_bridge()
    transport.deliver(
        bridge.cmd_topic,
        make_command("cmd_0006", "emergency_stop"),
        qos=1,
    )
    acks = published_on(transport, bridge.ack_topic)
    assert len(acks) == 1
    assert acks[0]["accepted"] is True
    assert acks[0]["reason"] == "emergency_stop"
    assert acks[0]["mode"] == DeviceMode.E_STOP
def test_device_mismatch_invalid() -> None:
    bridge, transport, _ = make_bridge()
    payload = make_command("cmd_0007", "pause")
    payload["device_id"] = "OTHER-ARM"
    transport.deliver(bridge.task_topic, payload, qos=1)
    assert bridge.counter["commands_invalid"] == 1
    assert published_on(transport, bridge.ack_topic) == []
def test_invalid_envelope_ignored() -> None:
    bridge, transport, _ = make_bridge()
    transport.deliver(bridge.task_topic, {"command_id": "broken"}, qos=1)
    assert bridge.counter["commands_invalid"] == 1
    assert bridge.counter["commands_processed"] == 0
def test_parse_via_device_command_from_dict() -> None:
    payload = dispatch_payload("tsk_0008")
    command = DeviceCommand.from_dict(payload)
    assert command.action == "dispatch"
    assert command.params["task_id"] == "tsk_0008"
def test_http_driver_pick_translates_response(monkeypatch) -> None:
    """The HTTP adapter must send one JSON action and parse arm fields."""
    captured: dict[str, object] = {}
    class FakeResponse:
        def __init__(self, body: bytes) -> None:
            self._body = body
        def __enter__(self) -> "FakeResponse":
            return self
        def __exit__(self, *exc_info: object) -> None:
            return None
```

```
        def read(self) -> bytes:
            return self._body
    def fake_urlopen(request: object, timeout: float) -> FakeResponse:
        captured["url"] = request.full_url  # type: ignore[attr-defined]
        captured["method"] = request.get_method()  # type: ignore[attr-defined]
        captured["body"] = json.loads(request.data.decode("utf-8"))  # type: ignore[attr-defined]
        assert timeout == 1.0
        return FakeResponse(
            json.dumps(
                {
                    "ok": True,
                    "mode": DeviceMode.IDLE,
                    "collected_weight": 0.12,
                    "review_result": "confirmed",
                    "evidence_url": "http://minio:9000/events/tasks/after.jpg",
                    "bins_after": {"foam": 0.12, "plastic": 0.0, "mixed": 0.0},
                }
            ).encode("utf-8")
        )
    monkeypatch.setattr("urllib.request.urlopen", fake_urlopen)
    driver = HttpArmDriver(
        "http://arm.local:8080/control",
        timeout=1.0,
        battery=88,
        bins={"foam": 0.0, "plastic": 0.0, "mixed": 0.0},
    )
    result = driver.pick("tsk_http_01", Position(119.6531, 26.3867), 2)
    assert captured["url"] == "http://arm.local:8080/control"
    assert captured["method"] == "POST"
    assert captured["body"] == {
        "action": "pick",
        "task_id": "tsk_http_01",
        "target": {"lng": 119.6531, "lat": 26.3867},
        "priority": 2,
    }
    assert result.ok is True
    assert result.collected_weight == 0.12
    assert result.evidence_url == "http://minio:9000/events/tasks/after.jpg"
    assert result.bins_after["foam"] == 0.12
def test_driver_registry_builds_simulated_and_http() -> None:
    assert "simulated" in ARM_DRIVER_REGISTRY
    assert "http" in ARM_DRIVER_REGISTRY
    assert isinstance(
        build_arm_driver({"backend": "simulated", "simulated": {"battery": 80}}),
        SimulatedArmDriver,
    )
    assert isinstance(
        build_arm_driver(
            {
                "backend": "http",
```

```
                "http": {"endpoint": "http://arm.local:8080/arm"},
            }
        ),
        HttpArmDriver,
    )
def test_driver_registry_builds_vendor_block() -> None:
    driver = build_arm_driver(
        {
            "backend": "dobot_example",
            "vendors": {
                "dobot_example": {
                    "kind": "http",
                    "endpoint": "http://192.168.1.101:8080/dobot",
                    "timeout": 2.0,
                    "battery": 77,
                }
            },
        }
    )
    assert isinstance(driver, HttpArmDriver)
    assert driver.endpoint == "http://192.168.1.101:8080/dobot"
    assert driver.timeout == 2.0
    assert driver.battery == 77
def test_driver_registry_rejects_unknown_backend() -> None:
    try:
        build_arm_driver({"backend": "missing-arm"})
    except ValueError as exc:
        assert "missing-arm" in str(exc)
    else:
        raise AssertionError("expected ValueError for unknown backend")
def test_hiwonder_driver_pick_plays_taught_sequence(monkeypatch) -> None:
    """The Hiwonder adapter must send torque, positions, then stop."""
    class FakeBoard:
        def __init__(self) -> None:
            self.torque: list[tuple[int, bool]] = []
            self.positions: list[tuple[float, list[list[int]]]] = []
            self.stops: list[list[int]] = []
            self.reception = 0
        def enable_reception(self, enable: bool = True) -> None:
            self.reception += int(enable)
        def bus_servo_enable_torque(self, servo_id: int, enable: bool) -> None:
            self.torque.append((servo_id, enable))
        def bus_servo_set_position(
            self, duration: float, positions: list[list[int]]
        ) -> None:
            self.positions.append((duration, positions))
        def bus_servo_stop(self, servo_ids: list[int]) -> None:
            self.stops.append(list(servo_ids))
        def get_battery(self) -> None:
            return None
```

```
    board = FakeBoard()


    monkeypatch.setattr(
        HiwonderBusServoArmDriver, "_new_board", lambda self: board
    )
    monkeypatch.setattr("time.sleep", lambda _seconds: None)
    driver = HiwonderBusServoArmDriver(
        serial_port="/dev/ttyAMA0",
        baudrate=1000000,
        servo_ids=[1, 2],
        home_positions=[[1, 500], [2, 500]],
        pick_sequence=[
            {"duration": 1.0, "positions": [[1, 650], [2, 500]]},
            {"duration": 1.2, "positions": [[1, 300], [2, 600]]},
        ],
        collected_weight=0.05,
    )
    result = driver.pick("tsk_hiwonder_01", Position(119.6531, 26.3867), 1)
    assert result.ok is True
    assert result.collected_weight == 0.05
    assert result.review_result == "recheck"
    assert result.bins_after["foam"] == 0.05
    assert board.reception == 1
    assert board.torque == [(1, True), (2, True)]
    assert board.positions == [
        (1.0, [[1, 650], [2, 500]]),
        (1.2, [[1, 300], [2, 600]]),
    ]
    assert board.stops == [[1, 2]]
def test_hiwonder_driver_loads_pick_sequence_file(tmp_path) -> None:
    seq_file = tmp_path / "pick_sequence.json"
    seq_file.write_text(
        json.dumps(
            {
                "format": "seasight_hiwonder_pick_sequence",
                "version": 1,
                "servo_ids": [1, 2],
                "steps": [
                    {"duration": 1.0, "positions": [[1, 650], [2, 500]]},
                    {"duration": 1.2, "positions": [[1, 300], [2, 600]]},
                ],
            }
        ),
        encoding="utf-8",
    )
    driver = HiwonderBusServoArmDriver(
        pick_sequence_file=str(seq_file),
        servo_ids=[1, 2],
    )
    assert driver.pick_sequence == [
        {"duration": 1.0, "positions": [[1, 650], [2, 500]]},
```

```
        {"duration": 1.2, "positions": [[1, 300], [2, 600]]},
    ]
def test_hiwonder_driver_rejects_both_sequence_sources() -> None:
    try:
        HiwonderBusServoArmDriver(
            pick_sequence=[
                {"duration": 1.0, "positions": [[1, 500]]}
            ],
            pick_sequence_file="whatever.json",
        )
    except ValueError as exc:
        assert "not both" in str(exc)
    else:
        raise AssertionError("expected ValueError for two sequence sources")
def test_teach_tool_output_loads_through_driver(tmp_path) -> None:
    from arm_bridge.tools.teach_hiwonder_sequence import build_output
    payload = build_output(
        [1, 2],
        [
            {"duration": 1.0, "positions": [[1, 650], [2, 500]]},
            {"duration": 1.0, "positions": [[1, 300], [2, 600]]},
        ],
        1.0,
    )
    out = tmp_path / "recorded.json"
    out.write_text(json.dumps(payload), encoding="utf-8")
    driver = HiwonderBusServoArmDriver(
        pick_sequence_file=str(out),
        servo_ids=[1, 2],
    )
    assert driver.pick_sequence == payload["steps"]
def test_hiwonder_driver_registry_builds_from_vendor_block() -> None:
    assert "hiwonder_bus_servo" in ARM_DRIVER_REGISTRY
    driver = build_arm_driver(
        {
            "backend": "hiwonder_bus_servo",
            "vendors": {
                "hiwonder_bus_servo": {
                    "kind": "hiwonder_bus_servo",
                    "serial_port": "/dev/ttyAMA0",
                    "baudrate": 1000000,
                    "servo_ids": [1, 2, 3, 4],
                    "home_positions": [[1, 500], [2, 500]],
                    "pick_sequence": [
                        {"duration": 1.0, "positions": [[1, 650], [2, 500]]}
                    ],
                }
            },
        }
    )
```

```
    assert isinstance(driver, HiwonderBusServoArmDriver)



    assert driver.servo_ids == [1, 2, 3, 4]
def test_hiwonder_status_without_sdk_is_deterministic(monkeypatch) -> None:
    def missing_sdk(self: HiwonderBusServoArmDriver) -> object:
        raise DriverError("ros_robot_controller_sdk is unavailable")
    monkeypatch.setattr(HiwonderBusServoArmDriver, "_new_board", missing_sdk)
    driver = HiwonderBusServoArmDriver(
        pick_sequence=[{"duration": 1.0, "positions": [[1, 500], [2, 500]]}],
        battery=90,
    )
    status = driver.status()
    assert status.mode == DeviceMode.IDLE
    assert status.battery == 90
    assert status.bins == {"foam": 0.0, "plastic": 0.0, "mixed": 0.0}
    try:
        driver.pick("tsk_hiwonder_02", Position(119.6531, 26.3867), 1)
    except DriverError:
        pass
    else:
        raise AssertionError("expected DriverError when SDK is missing")
def test_check_driver_resolves_without_calling() -> None:
    from arm_bridge.main import check_driver
    cfg = {
        "driver": {
            "backend": "dobot_example",
            "vendors": {
                "dobot_example": {
                    "kind": "http",
                    "endpoint": "http://192.168.1.101:8080/dobot",
                }
            },
        }
    }
    assert check_driver(cfg, None) == 0
    assert check_driver(cfg, "simulated") == 0
    assert check_driver(cfg, "missing-arm") == 1
# ===== edge/arm_bridge/test_ros_driver.py =====
"""RosArmDriver 的回归测试（不需要真机/ROS）。
覆盖:
  1. 惰性注册：import 才进注册表，且必须进同一份注册表
     （踩过的坑：ros_driver 用 `from drivers import` 时会注册到另一份
     模块对象，导致配置选 ros_arm_control 报 KeyError）
  2. 无 ROS 环境下必须优雅降级，不能抛异常
  3. /joint_states 读数 → ArmStatus.servos 的转换（键类型与串口路径一致）
  4. 口径：target 是 GPS 不是关节角
"""
from __future__ import annotations
import math
import sys
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parent.parent.parent


sys.path.insert(0, str(ROOT / "edge"))
sys.path.insert(0, str(ROOT / "edge" / "arm_bridge"))
from arm_bridge.drivers import (  # noqa: E402
    ARM_DRIVER_REGISTRY,
    DeviceMode,
    Position,
)
import arm_bridge.ros_driver as ros_driver  # noqa: E402
# ① 注册表
def test_registers_under_expected_name() -> None:
    assert "ros_arm_control" in ARM_DRIVER_REGISTRY
    assert ros_driver.RosArmDriver is ARM_DRIVER_REGISTRY["ros_arm_control"]
def test_registered_into_the_same_registry_object() -> None:
    """必须注册进 bridge 用的那一份，不能是同名但不同身份的模块。
    这是真实踩过的坑：`from drivers import` 与`from arm_bridge.drivers
    import` 会产生两份 ARM_DRIVER_REGISTRY，配置选 ros_arm_control 时
    报 KeyError。
    不能用 isinstance 判断：ArmDriver 是 Protocol 但没加
      @runtime_checkable，isinstance 会抛 TypeError。改为逐方法核对。
    """
    inst = ARM_DRIVER_REGISTRY["ros_arm_control"]()
    for name in ("status", "pick", "pause", "resume",
                 "return_home", "emergency_stop"):
        assert callable(getattr(inst, name, None)), name
def test_other_drivers_still_present() -> None:
    """新增驱动不能挤掉原有的三个。"""
    for name in ("simulated", "http", "hiwonder_bus_servo"):
        assert name in ARM_DRIVER_REGISTRY, name
# ② 无 ROS 时降级
def test_status_degrades_without_ros() -> None:
    d = ros_driver.RosArmDriver()
    st = d.status()          # 不应抛
    assert st.servos == {}
    assert st.mode == DeviceMode.IDLE
def test_pick_without_ros_raises_readable_error() -> None:
    """真要用时必须有可执行的错误信息，不能是 ImportError 堆栈。"""
    d = ros_driver.RosArmDriver()
    try:
        d.pick("T-1", Position(119.65, 26.38), 3)
    except RuntimeError as exc:
        assert "rospy" in str(exc) or "ROS" in str(exc), str(exc)
    except Exception as exc:  # noqa: BLE001
        raise AssertionError("应为 RuntimeError，实际 %r" % exc) from exc
    # 若本机恰好装了 rospy 则会走真实分支，这里不断言
def test_emergency_stop_sets_mode_without_ros() -> None:
    d = ros_driver.RosArmDriver()
    d.emergency_stop()
    assert d.status().mode == DeviceMode.E_STOP
# ③ joint_states 转换
```

```
def test_joint_snapshot_maps_to_servo_dict() -> None:
    """键必须是舵机序号（字符串），与 HiwonderBusServo 路径的键类型一致——
    页面的展示逻辑不该为两种驱动写两套。"""
    snap = ros_driver._JointSnapshot(
        names=["joint1", "joint2", "joint3", "joint4", "joint5", "r_joint"],
        positions=[0.0, 0.5236, -1.3614, -1.7593, 0.0, -1.7802],
        velocities=[0.0] * 6,
        efforts=[0.0] * 6,
    )
    out = snap.as_servo_map()
    assert set(out) == {"1", "2", "3", "4", "5", "6"}
    assert out["2"]["position"] == 0.5236
    assert out["2"]["joint"] == "joint2"
    # 度数换算正确
    assert abs(out["2"]["position_deg"] - 30.0) < 0.01
    assert abs(out["4"]["position_deg"] - (-100.8)) < 0.01
def test_snapshot_handles_missing_arrays() -> None:
    """有些消息里 velocity/effort 是空数组，不能 IndexError。"""
    snap = ros_driver._JointSnapshot(
        names=["joint1"], positions=[0.5], velocities=[], efforts=[]
    )
    out = snap.as_servo_map()
    assert out["1"]["position"] == 0.5
    assert "velocity" not in out["1"]
def test_snapshot_with_fewer_joints_than_names() -> None:
    """positions 比 names 短时不能越界。"""
    snap = ros_driver._JointSnapshot(
        names=["joint1", "joint2", "joint3"], positions=[0.1, 0.2]
    )
    out = snap.as_servo_map()
    assert len(out) == 2
    assert out["2"]["joint"] == "joint2"
# ④ 默认值与口径
def test_default_home_matches_vendor_calibration() -> None:
    """默认 home 位应与实测 /joint_states 的厂商标定值一致。
    那组值全是整数度（30 / 78 / 100.8 / 102），是厂商标定过的 home 位。
    """
    degs = [round(math.degrees(x), 1) for x in ros_driver.DEFAULT_HOME_RAD]
    assert degs == [0.0, 30.0, -78.0, -100.8, 0.0, -102.0]
def test_joint_names_cover_six_dof() -> None:
    assert len(ros_driver.DEFAULT_JOINT_NAMES) == 6
    assert "r_joint" in ros_driver.DEFAULT_JOINT_NAMES
def test_pick_treats_target_as_gps_not_joint() -> None:
    """口径守护：target 是 GPS 目标点，不能当关节角用。
    逆运动学在厂商 ROS 栈里（armpi_fpv_kinematics），我们不解。
    这个测试存在的意义是：若将来有人把 target 直接当关节角下发，
    单位会错得离谱（GPS 是 119.65，关节角是弧度）。
    """
    d = ros_driver.RosArmDriver()
    lng, lat = 119.6521, 26.3864
```

```
    assert abs(lng) > 90, "这是经度，不是关节角"



    assert abs(lat) <= 90
    # 驱动不应把 target 存成关节角
    assert not hasattr(d, "target_joint_rad")
    assert d.home.lng != lng or d.home.lat != lat
if __name__ == "__main__":
    import pytest
    sys.exit(pytest.main([__file__, "-v"]))
# ===== edge/arm_bridge/test_hiwonder_vendor_format.py =====
"""厂商示教格式与遥测的回归测试。
覆盖两个真实缺陷（都在 2026-10-05 精读厂商资料后才发现）：
1. _load_sequence_file 原本只接受 {"duration","positions"} 格式，
   而厂商示教程序 案例5 示教记录实现/bus_servo_record.py 写出的是
   扁平的位置列表 [1000, 940]：直接喂进去会解析失败。
2. status() 调 Board.get_battery() 只调一次，而该方法是非阻塞的
   （队列空即返回 None），所以 battery 永远是构造时的初值：一个
   "声明了却没实现"的静默缺陷。
这些用假 Board 验证，不需要真机。
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path
ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from arm_bridge.drivers import (  # noqa: E402
    DeviceMode,
    HiwonderBusServoArmDriver,
    Position,
)
class FakeBoard:
    """模拟厂商 SDK 的 Board，只实现我们用到的部分。"""
    def __init__(self, *, battery_vin: int | None = 7400,
                 servo_values: dict[int, dict[str, int]] | None = None,
                 battery_empty_first: int = 0) -> None:
        self._battery_vin = battery_vin
        self._servo_values = servo_values or {}
        # 前 N 次 get_battery 返回 None（模拟异步队列尚未收到回包）
        self._battery_empty_first = battery_empty_first
        self._battery_calls = 0
        self.reception = False
        self.stopped: list[int] = []
        self.torque: list[tuple[int, bool]] = []
        self.moves: list[tuple[float, list]] = []
    # --- 我们驱动用到的 ---
    def enable_reception(self, on: bool = True) -> None:
        self.reception = on
    def get_battery(self):
        self._battery_calls += 1
```

```
        if self._battery_calls <= self._battery_empty_first:
            return None
        return self._battery_vin
    def bus_servo_read_vin(self, sid):
        return [self._servo_values.get(sid, {}).get("vin", 11000)]
    def bus_servo_read_temp(self, sid):
        return [self._servo_values.get(sid, {}).get("temp", 42)]
    def bus_servo_read_position(self, sid):
        return [self._servo_values.get(sid, {}).get("position", 512)]
    def bus_servo_enable_torque(self, sid, on):
        self.torque.append((sid, on))
    def bus_servo_set_position(self, duration, positions):
        self.moves.append((duration, positions))
    def bus_servo_stop(self, sids):
        self.stopped.extend(sids)
def _driver(board: FakeBoard, **kw) -> HiwonderBusServoArmDriver:
    """构造一个已注入 FakeBoard 的驱动，绕过真实串口。"""
    d = HiwonderBusServoArmDriver(
        pick_positions=[[1, 500]],
        **kw,
    )
    d._board = board
    return d
# ① 厂商扁平格式的序列文件
def test_vendor_flat_position_file_is_accepted(tmp_path: Path) -> None:
    """厂商示教程序写出的 [1000, 940] 必须能被直接回放。"""
    f = tmp_path / "servo_positions.json"
    f.write_text(json.dumps([1000, 940]), encoding="utf-8")
    seq = HiwonderBusServoArmDriver._load_sequence_file(str(f))
    assert len(seq) == 2, seq
    assert seq[0]["positions"] == [[1, 1000]]
    assert seq[1]["positions"] == [[1, 940]]
def test_vendor_flat_file_with_steps_wrapper(tmp_path: Path) -> None:
    """{'steps': [1000, 940]} 这种包装也要能吃。"""
    f = tmp_path / "wrapped.json"
    f.write_text(json.dumps({"steps": [1000, 940]}), encoding="utf-8")
    seq = HiwonderBusServoArmDriver._load_sequence_file(str(f))
    assert len(seq) == 2
def test_our_own_format_still_works(tmp_path: Path) -> None:
    """我们自己的 step 格式不能被这次改动破坏。"""
    f = tmp_path / "ours.json"
    f.write_text(
        json.dumps([{"duration": 1.5, "positions": [[1, 500], [2, 480]]}]),
        encoding="utf-8",
    )
    seq = HiwonderBusServoArmDriver._load_sequence_file(str(f))
    assert seq[0]["duration"] == 1.5
    assert seq[0]["positions"] == [[1, 500], [2, 480]]
# ② 电池遥测：非阻塞队列需要重试
def test_battery_read_retries_until_queue_fills() -> None:
```

```
    """get_battery 前几次返回 None（队列空），重试后应拿到真实电压。"""


    board = FakeBoard(battery_vin=7400, battery_empty_first=3)
    d = _driver(board, battery=100)
    st = d.status()
    # 7400mV 落在 (6000, 8400] 区间 → 中段电量；关键不是 100（初值）
    assert st.battery != 100, "battery 应来自真实电压而非构造初值"
    assert 0 <= st.battery <= 100
    assert board._battery_calls >= 4, "应至少重试到队列有数据"
def test_battery_handles_permanent_empty_queue() -> None:
    """队列始终空时不能抛异常，battery 退回初值。"""
    board = FakeBoard(battery_vin=None, battery_empty_first=999)
    d = _driver(board, battery=88)
    st = d.status()
    assert st.battery == 88
# ③ 舵机遥测（证据链）
def test_status_reports_per_servo_telemetry() -> None:
    """电压/温度/位置要能读出来：这是"真机真的动了"的硬证据。"""
    board = FakeBoard(
        battery_vin=7400,
        servo_values={1: {"vin": 11500, "temp": 38, "position": 620}},
    )
    d = _driver(board, servo_ids=[1, 2])
    st = d.status()
    assert "1" in st.servos
    assert st.servos["1"]["position"] == 620
    assert st.servos["1"]["temp"] == 38
    assert st.servos["1"]["vin"] == 11500
    # 舵机 2 没给数据 → 用了 SDK 默认值，仍应有条目
    assert "2" in st.servos
def test_status_without_board_is_safe() -> None:
    """从未连过真机时 status 不能抛异常。"""
    d = HiwonderBusServoArmDriver(pick_positions=[[1, 500]])
    d._board = None
    st = d.status()
    assert st.battery == d.battery
    assert st.servos == {}
def test_telemetry_failure_does_not_break_status() -> None:
    """某个舵机读失败时，其余仍要能返回（总线回读是异步的）。"""
    class FlakyBoard(FakeBoard):
        def bus_servo_read_temp(self, sid):
            if sid == 1:
                raise RuntimeError("timeout")
            return [45]
    board = FlakyBoard(battery_vin=7400)
    d = _driver(board, servo_ids=[1, 2])
    st = d.status()
    assert "temp" not in st.servos.get("1", {})
    assert st.servos.get("2", {}).get("temp") == 45
# ④ 驱动契约
def test_driver_still_satisfies_protocol() -> None:
```

```
    """新增字段不能破坏 ArmDriver 协议的实现完整性。"""


    d = HiwonderBusServoArmDriver(pick_positions=[[1, 500]])
    for name in ("pick", "pause", "resume", "return_home", "emergency_stop", "status"):
        assert callable(getattr(d, name)), name
def test_emergency_stop_then_status_reports_mode() -> None:
    """急停后 status 要能看到状态变化。"""
    board = FakeBoard()
    d = _driver(board)
    d.emergency_stop()
    assert d.status().mode == DeviceMode.E_STOP
if __name__ == "__main__":
    import pytest
    sys.exit(pytest.main([__file__, "-v"]))
# ===== ml/scripts/calibrate_camera.py =====
#!/usr/bin/env python3
"""探海灵眸 Oceanus — 相机标定与平面映射工具。
功能:
  1. `chessboard`: 棋盘格内参标定 (cv2.calibrateCamera) + 畸变系数 +
                   简单平面映射 (棋盘角点像素↔米制平面单应矩阵)。
  2. `plane-map`: 由 ≥4 组显式对应点计算平面单应矩阵 (cv2.findHomography)。
纪律 (冻结):
  - 本工具是 OpenCV 传统视觉标定，不是 YOLO、不是视觉大模型、不是已训练深度模型。
  - 没有可用图像时明确失败 (退出码 1)，不输出任何默认假参数。
  - 合成棋盘图 evidence_level 最高 E2; E3 需要 `--real-source` (真实采集说明);
    E4 一律拒绝 (脚本不自动授予)。
  - 报告固定包含: 命令、日期、代码版本、输入文件哈希、图像数、可用/跳过数、
    跳过原因、重投影误差、相机矩阵、畸变系数、单应矩阵、evidence_level。
退出码:
  0 = 标定成功
  1 = 标定失败 (没有图像 / 可用图像不足 / 无法收敛) ——不写输出文件
  2 = 用法或配置错误 (pattern 非法、对应点不足、E4 越级等)
  3 = 依赖缺失 (opencv-python / numpy)
用法:
  python ml/scripts/calibrate_camera.py chessboard \
      --images <dir_or_glob> --pattern 9x6 --square-size 0.03 \
      --output artifacts/calibration/cam_01.json
  python ml/scripts/calibrate_camera.py plane-map \
      --points correspondences.json --output artifacts/calibration/plane_01.json
"""
from __future__ import annotations
import argparse
import glob
import json
import sys
from datetime import datetime, timezone
from pathlib import Path
from typing import Any
THIS_FILE = Path(__file__).resolve()
ML = THIS_FILE.parent.parent
ROOT = ML.parent
```

```
CODE_TAG = "seasight-wp13-20260919"



IMG_SUFFIX = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
# evidence 纪律 (与 docs/perception-data-protocol.md §4 一致)
ALLOWED_CALIBRATION_EVIDENCE = ("E1", "E2", "E3")  # E3 需 --real-source
class DependencyError(Exception):
    pass
class ConfigError(Exception):
    pass
class CalibrationError(Exception):
    pass
def _check_dependencies() -> dict[str, str]:
    versions: dict[str, str] = {}
    missing: list[str] = []
    try:
        import cv2  # type: ignore[import-not-found]
        versions["cv2"] = cv2.__version__
    except ImportError:
        missing.append("opencv-python-headless (或 opencv-python) , 提供 cv2")
    try:
        import numpy  # type: ignore[import-not-found]
        versions["numpy"] = numpy.__version__
    except ImportError:
        missing.append("numpy")
    if missing:
        raise DependencyError("缺少依赖: " + "、".join(missing) + "。请先安装再重跑。")
    return versions
def gather_images(source: str) -> list[Path]:
    """解析 --images: 目录 → 递归扫描; 含通配符 → glob; 否则按单文件。"""
    p = Path(source)
    if p.is_dir():
        return sorted(f for f in p.rglob("*") if f.is_file() and f.suffix.lower() in IMG_SUFFIX)
    if any(ch in source for ch in "*?["):
        return sorted(
            Path(f) for f in glob.glob(source) if Path(f).suffix.lower() in IMG_SUFFIX
        )
    if p.is_file() and p.suffix.lower() in IMG_SUFFIX:
        return [p]
    return []
def parse_pattern(text: str) -> tuple[int, int]:
    """'9x6' / '9,6' → (9, 6) (内角点列×行) 。"""
    parts = text.lower().replace(",", "x").split("x")
    if len(parts) != 2:
        raise ConfigError(f"pattern 应为 <列>x<行>, 如 9x6, 实际 {text!r}")
    try:
        cols, rows = (int(v) for v in parts)
    except ValueError:
        raise ConfigError(f"pattern 含非数字: {text!r}") from None
    if cols < 2 or rows < 2:
        raise ConfigError(f"pattern 至少 2x2, 实际 {text!r}")
    return cols, rows
```

```
def _sha256(path: Path) -> str:
    import hashlib
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()
def find_chessboard_corners(gray, pattern: tuple[int, int]):
    """返回 (cols, rows) 内角点；找不到返回 None。"""
    import cv2
    ret, corners = cv2.findChessboardCorners(gray, pattern, None)
    if not ret:
        return None
    return corners
def calibrate_chessboard(
    image_paths: list[Path],
    pattern: tuple[int, int],
    square_size_m: float,
    evidence_level: str = "E1",
    real_source: str = "",
    max_images: int | None = None,
) -> dict[str, Any]:
    """棋盘格内参 + 畸变 + 平面映射标定。
    没有可用图像（或可用 < 2）→ 抛 CalibrationError，绝不返回默认假参数。
    """
    import cv2
    import numpy as np
    if evidence_level not in ALLOWED_CALIBRATION_EVIDENCE:
        raise ConfigError(
            f"evidence_level={evidence_level!r} 非法；标定允许 {ALLOWED_CALIBRATION_EVIDENCE}"
            "（合成最高 E2；E3 需 --real-source；脚本不授予 E4）"
        )
    if evidence_level == "E3" and not real_source.strip():
        raise ConfigError("E3 需要 --real-source（真实采集说明），否则只能标 E1/E2")
    if square_size_m <= 0:
        raise ConfigError(f"square_size 必须为正数，实际 {square_size_m!r}")
    if max_images is not None:
        image_paths = image_paths[:max_images]
    paths = [p for p in image_paths if p.is_file()]
    if not paths:
        raise CalibrationError(
            f"没有可用标定图像（{len(image_paths)} 个候选均不可读）。"
            "标定必须基于真实图像，拒绝输出默认假参数。"
        )
    cols, rows = pattern
    objp = np.zeros((cols * rows, 3), np.float32)
    objp[:, :2] = np.mgrid[0:cols, 0:rows].T.reshape(-1, 2) * square_size_m
    obj_points: list[np.ndarray] = []
    img_points: list[np.ndarray] = []
    used: list[dict[str, Any]] = []
```

```
skipped: list[dict[str, Any]] = []



for p in paths:
    gray = cv2.imread(str(p), cv2.IMREAD_GRAYSCALE)
    if gray is None:
        skipped.append({"path": str(p), "reason": "无法读取为灰度图"})
        continue
    corners = find_chessboard_corners(gray, pattern)
    if corners is None:
        skipped.append({"path": str(p), "reason": f"未检测到 {cols}x{rows} 棋盘内角点"})
        continue
    obj_points.append(objp)
    img_points.append(corners)
    used.append({"path": str(p), "corners": int(corners.shape[0])})
if len(used) < 2:
    raise CalibrationError(
        f"可用标定图像 {len(used)} 张 < 2，无法求内参。"
        "标定必须基于足够多视图，拒绝输出默认假参数。"
    )
try:
    rms, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
        obj_points, img_points, gray.shape[::-1], None, None
    )
except cv2.error as e:  # type: ignore[attr-defined]
    raise CalibrationError(f"cv2.calibrateCamera 失败（无法收敛）：{e}") from e
per_image_errors: list[dict[str, Any]] = []
for i, (p, rvec, tvec) in enumerate(zip(used, rvecs, tvecs)):
    proj, _ = cv2.projectPoints(obj_points[i], rvec, tvec, mtx, dist)
    err = float(cv2.norm(img_points[i], proj, cv2.NORM_L2) / len(proj))
    per_image_errors.append({"image": p["path"], "reprojection_error_px": round(err, 6)})
# 简单平面映射：取重投影误差最小的视图，像素角点 ↔ 米制平面角点
homography = None
plane_residual = None
if per_image_errors:
    best = min(range(len(per_image_errors)), key=lambda i: per_image_errors[i]["reprojection_er…
    H, res = compute_plane_homography(
        img_points[best].reshape(-1, 2).tolist(),
        obj_points[best][:, :2].tolist(),
    )
    homography = [row for row in H]
    plane_residual = res
report = {
    "status": "calibrated",
    "command": "calibrate_camera.py chessboard",
    "date": datetime.now(timezone.utc).isoformat(),
    "code_tag": CODE_TAG,
    "evidence_level": evidence_level,
    "real_source": real_source if real_source.strip() else None,
    "pattern": {"cols": cols, "rows": rows},
    "square_size_m": square_size_m,
    "input_files": [{"path": u["path"], "sha256": _sha256(Path(u["path"]))} for u in used],
```

```
        "image_count": len(paths),
        "used_images": used,
        "skipped_images": skipped,
        "rms_error_px": round(float(rms), 6),
        "camera_matrix": [[float(v) for v in row] for row in mtx.tolist()],
        "dist_coeffs": [float(v) for v in dist.ravel()],
        "per_image_errors": per_image_errors,
        "homography": homography,
        "plane_residual": plane_residual,
    }
    return report
def compute_plane_homography(src_pts: list[list[float]], dst_pts: list[list[float]]) -> tuple[list[…
    """像素坐标 ↔ 平面坐标单应矩阵（cv2.findHomography，最小二乘）。
    返回 (H 3x3, 残差报告)。对应点 < 4 抛 ConfigError。
    """
    import cv2
    import numpy as np
    src = np.asarray(src_pts, dtype=np.float32)
    dst = np.asarray(dst_pts, dtype=np.float32)
    if src.shape != dst.shape or src.shape[0] < 4:
        raise ConfigError(
            f"平面映射至少需要 4 组对应点，实际 {src.shape[0] if src.ndim == 2 else 0} 组"
        )
    H, _mask = cv2.findHomography(src, dst, method=0)  # 最小二乘（非 RANSAC）
    proj = cv2.perspectiveTransform(src.reshape(-1, 1, 2), H).reshape(-1, 2)
    errs = np.linalg.norm(proj - dst, axis=1)
    return (
        [[float(v) for v in row] for row in H.tolist()],
        {
            "count": int(len(errs)),
            "unit": "plane",  # 残差位于目标坐标空间（平面映射时为米制）
            "mean": round(float(errs.mean()), 6),
            "max": round(float(errs.max()), 6),
        },
    )
def plane_map_from_points(points: list[dict[str, Any]]) -> dict[str, Any]:
    """由显式对应点计算平面单应矩阵；不需要图像。"""
    pairs = []
    for i, pt in enumerate(points):
        pix = pt.get("pixel")
        pla = pt.get("plane")
        if not (isinstance(pix, (list, tuple)) and len(pix) == 2
                and isinstance(pla, (list, tuple)) and len(pla) == 2):
            raise ConfigError(f"对应点 {i} 必须包含 pixel=[x,y] 与 plane=[X,Y]")
        pairs.append(([float(v) for v in pix], [float(v) for v in pla]))
    if len(pairs) < 4:
        raise ConfigError(f"平面映射至少需要 4 组对应点，实际 {len(pairs)} 组")
    H, res = compute_plane_homography([p[0] for p in pairs], [p[1] for p in pairs])
    return {
        "status": "calibrated",
```

```
        "command": "calibrate_camera.py plane-map",
        "date": datetime.now(timezone.utc).isoformat(),
        "code_tag": CODE_TAG,
        "homography": H,
        "residual": res,
        "point_count": len(pairs),
        "input_files": [],
    }
def _load_points(path: str) -> list[dict[str, Any]]:
    p = Path(path)
    if not p.is_file():
        raise ConfigError(f"找不到对应点文件 {p}")
    try:
        data = json.loads(p.read_text(encoding="utf-8"))
    except json.JSONDecodeError as e:
        raise ConfigError(f"{p} 不是合法 JSON: {e}") from e
    pts = data.get("points") if isinstance(data, dict) else data
    if not isinstance(pts, list):
        raise ConfigError(f'{p} 需要 {{"points": [...]}} 结构')
    return pts
def _print_report(r: dict[str, Any]) -> None:
    print(f"状态       : {r['status']}")
    print(f"evidence_level  : {r.get('evidence_level')}")
    if r.get("real_source"):
        print(f"real_source     : {r['real_source']}")
    if r.get("pattern"):
        print(f"棋盘格      : {r['pattern']['cols']}x{r['pattern']['rows']} 内角点, "
              f"方格 {r['square_size_m']} m")
    print(f"图像数      : {r.get('image_count')}, 可用 {len(r.get('used_images', []))}, "
          f"跳过 {len(r.get('skipped_images', []))}")
    for s in r.get("skipped_images", []):
        print(f"  - 跳过 {s['path']}: {s['reason']}")
    if r.get("rms_error_px") is not None:
        print(f"重投影误差 RMS  : {r['rms_error_px']} px")
    if r.get("camera_matrix"):
        print(f"相机矩阵     : {r['camera_matrix']}")
    if r.get("dist_coeffs"):
        print(f"畸变系数     : {r['dist_coeffs']}")
    if r.get("per_image_errors"):
        worst = max(r["per_image_errors"], key=lambda e: e["reprojection_error_px"])
        print(f"单图误差范围   : 最大 {worst['reprojection_error_px']} px ({worst['image']})")
    if r.get("homography"):
        print(f"平面单应矩阵  : {r['homography']}")
    if r.get("plane_residual"):
        print(f"平面映射残差  : {r['plane_residual']}")
    if r.get("residual"):
        print(f"平面映射残差  : {r['residual']}")
    print(f"输入文件哈希  : {[f['sha256'][:12] for f in r.get('input_files', [])]}")
def main(argv: list[str] | None = None) -> int:
    try:
```

```
        versions = _check_dependencies()


    except DependencyError as e:
        print(f"[错误] {e}", file=sys.stderr)
        return 3
    ap = argparse.ArgumentParser(
        prog="calibrate_camera",
        description="Oceanus 相机标定（OpenCV 传统视觉，非训练模型）",
    )
    sub = ap.add_subparsers(dest="command", required=True)
    p_cb = sub.add_parser("chessboard", help="棋盘格内参+畸变+平面映射")
    p_cb.add_argument("--images", required=True, help="图像目录 / glob / 单文件")
    p_cb.add_argument("--pattern", default="9x6", help="内角点 列x行，如 9x6")
    p_cb.add_argument("--square-size", type=float, default=0.03, help="方格边长（米）")
    p_cb.add_argument("--output", required=True, help="标定报告 JSON 输出路径")
    p_cb.add_argument("--evidence-level", default="E1", help="E1|E2|E3（E3 需 --real-source）")
    p_cb.add_argument("--real-source", default="", help="真实采集说明（E3 必填）")
    p_cb.add_argument("--max-images", type=int, default=None)
    p_cb.set_defaults(func=_cmd_chessboard)
    p_pm = sub.add_parser("plane-map", help="由对应点计算平面单应矩阵")
    p_pm.add_argument("--points", required=True, help="对应点 JSON: {\"points\":[{pixel,plane},...]…
    p_pm.add_argument("--output", required=True)
    p_pm.add_argument("--evidence-level", default="E1")
    p_pm.add_argument("--real-source", default="")
    p_pm.set_defaults(func=_cmd_plane_map)
    args = ap.parse_args(argv)
    return args.func(args, versions)
def _write_output(path: str, report: dict[str, Any]) -> None:
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
def _cmd_chessboard(args: argparse.Namespace, versions: dict[str, str]) -> int:
    try:
        pattern = parse_pattern(args.pattern)
    except ConfigError as e:
        print(f"[错误] {e}", file=sys.stderr)
        return 2
    images = gather_images(args.images)
    if not images:
        print(
            "[错误] 没有找到任何标定图像。标定必须基于真实图像，"
            "拒绝输出默认假参数。",
            file=sys.stderr,
        )
        return 1
    try:
        report = calibrate_chessboard(
            images,
            pattern,
            args.square_size,
            evidence_level=args.evidence_level,
```

```
            real_source=args.real_source,
            max_images=args.max_images,
        )
    except CalibrationError as e:
        print(f"[错误] {e}", file=sys.stderr)
        print(f"[错误] 未写入 {args.output}（不输出假参数）", file=sys.stderr)
        return 1
    except ConfigError as e:
        print(f"[错误] {e}", file=sys.stderr)
        return 2
    _write_output(args.output, report)
    print(f"依赖版本     : {versions}")
    _print_report(report)
    print(f"输出       : {args.output}")
    return 0
def _cmd_plane_map(args: argparse.Namespace, versions: dict[str, str]) -> int:
    try:
        if args.evidence_level not in ALLOWED_CALIBRATION_EVIDENCE:
            raise ConfigError(
                f"evidence_level={args.evidence_level!r} 非法；允许 {ALLOWED_CALIBRATION_EVIDENCE}"
            )
        if args.evidence_level == "E3" and not args.real_source.strip():
            raise ConfigError("E3 需要 --real-source（真实采集说明）")
        points = _load_points(args.points)
        report = plane_map_from_points(points)
    except ConfigError as e:
        print(f"[错误] {e}", file=sys.stderr)
        return 2
    report["evidence_level"] = args.evidence_level
    report["real_source"] = args.real_source if args.real_source.strip() else None
    _write_output(args.output, report)
    print(f"依赖版本     : {versions}")
    _print_report(report)
    print(f"输出       : {args.output}")
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```